10 PROBLEM STATEMENTS IN PYTHON

Case Study Documentation

Each case study is prepared in the required format: problem statement, objectives, assumptions, algorithm, flowchart, pseudocode, Python source code, test cases, output screenshots, and conclusion.

List of Case Studies

1. Attendance Recovery Computation (Case Study 12)
2. Advanced Numeric Pattern Generator (Case Study 27)
3. Password Policy Validation Engine (Case Study 29)
4. Text Compression Using Character Runs (Case Study 35)
5. Warehouse Inventory Reconciliation (Case Study 14)
6. Repeated Transaction Detection (Case Study 28)
7. Student Email and Identifier Validation (Case Study 30)
8. Nested Loop Calendar Generator (Case Study 37)
9. Integrated University Fee and Eligibility Audit (Case Study 71)
10. Integrated Student Record Consistency Checker (Case Study 75)

Case Study 1: Attendance Recovery Computation

Case Study No. 12 | Difficulty: Easy

1. Problem Statement

Create a Python program that calculates a student's attendance percentage and determines how many additional classes the student must attend consecutively to reach the minimum required attendance of 75%.

2. Objectives

- Read the number of classes held and classes attended.
- Calculate current attendance percentage.
- Determine whether attendance is already at least 75%.
- If not, calculate the minimum additional classes that must be attended without missing any.

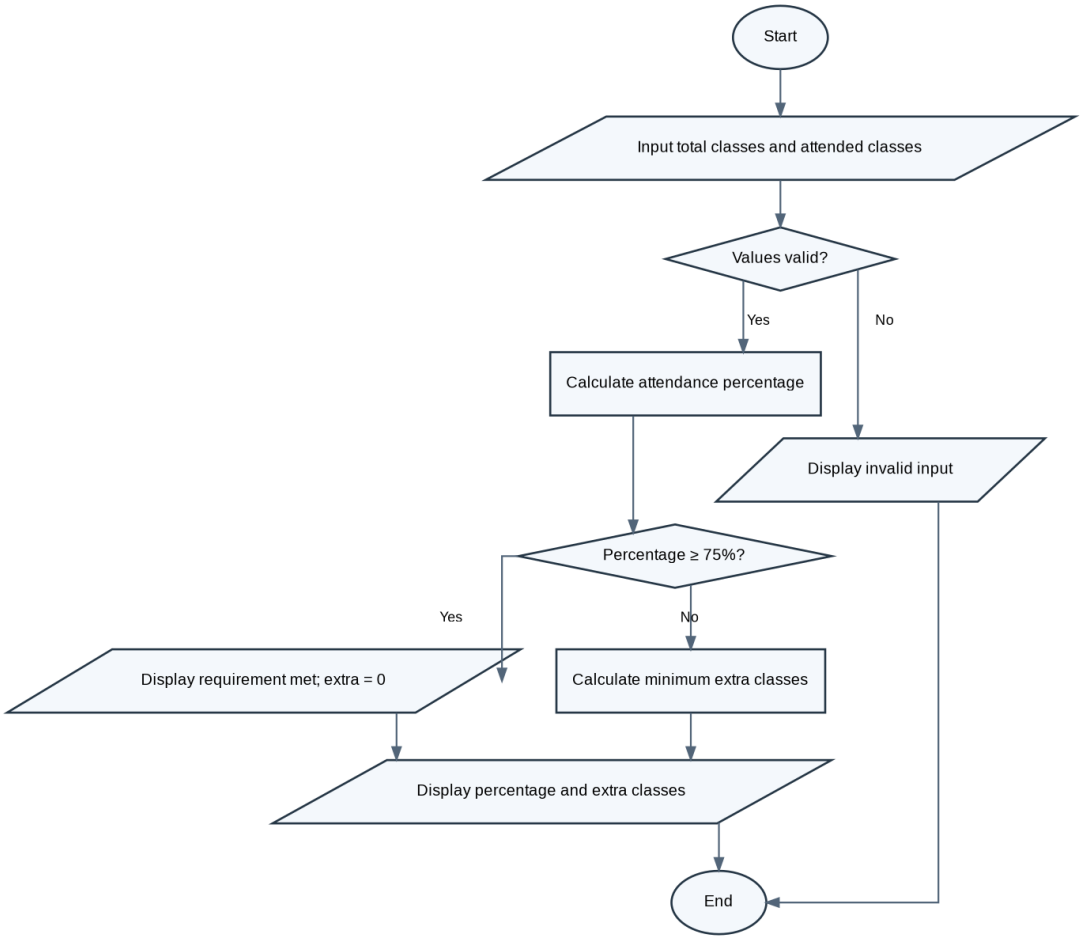
3. Assumptions

- Classes held must be greater than zero.
- Classes attended must be between zero and classes held.
- Every additional class is attended by the student.
- The required attendance threshold is 75%.

4. Algorithm

1. Start the program.
2. Input total classes held and classes attended.
3. Validate that the values are sensible.
4. Calculate percentage = attended / held × 100.
5. If percentage is at least 75, display that the requirement is met.
6. Otherwise find the smallest whole number x such that (attended + x) / (held + x) ≥ 0.75.
7. Display percentage and additional classes.
8. Stop.

5. Flowchart



6. Pseudocode

```
START
INPUT held, attended
IF held <= 0 OR attended < 0 OR attended > held THEN
    PRINT "Invalid attendance values"
ELSE
    percentage = attended / held * 100
    IF percentage >= 75 THEN
        extra = 0
    ELSE
        extra = 0
        WHILE (attended + extra) / (held + extra) < 0.75
            extra = extra + 1
        END WHILE
    END IF
    PRINT percentage, extra
END IF
STOP
```

7. Python Source Code

```
attended = int(input("Enter classes attended: "))
total = int(input("Enter total classes conducted: "))

if total <= 0 or attended < 0 or attended > total:
    print("Invalid attendance details.")
else:
    percentage = (attended / total) * 100
```

```
print(f"Current attendance: {percentage:.2f}%")
  required = 75

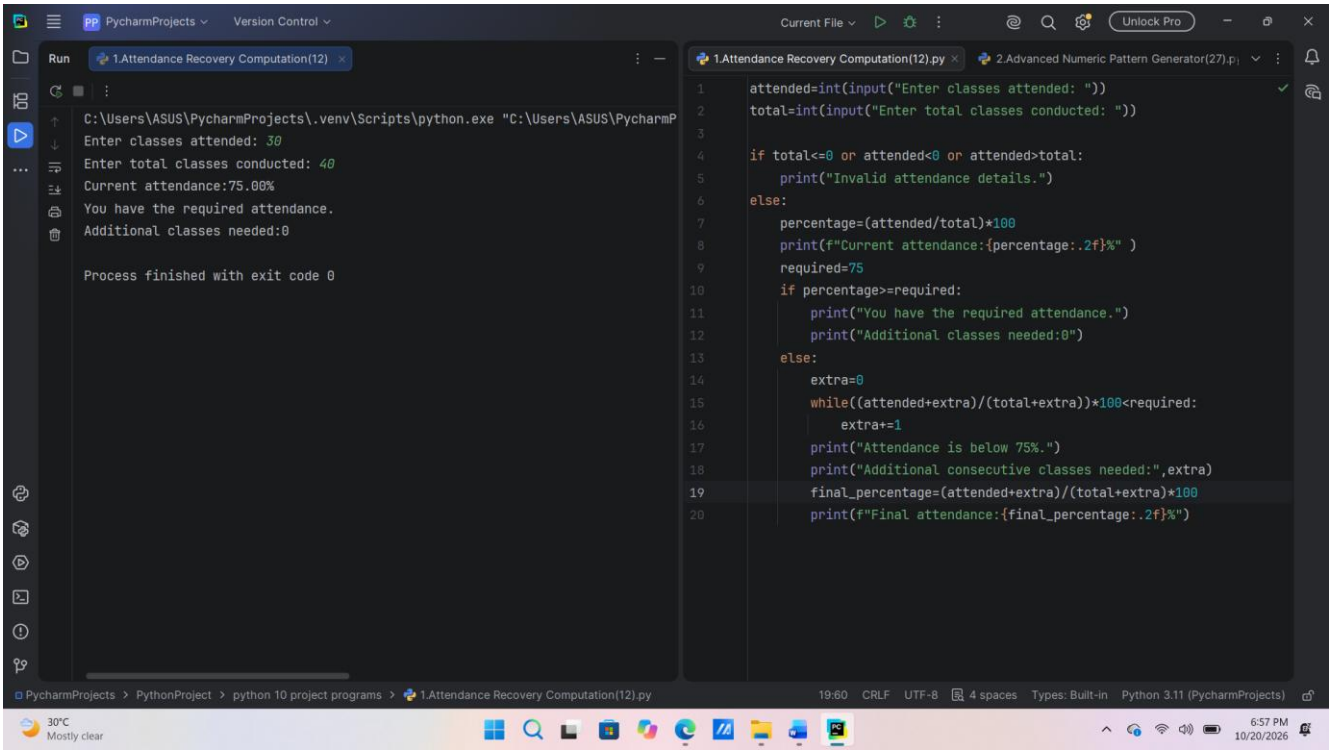
  if percentage >= required:
      print("You have the required attendance.")
      print("Additional classes needed: 0")
  else:
      extra = 0
      while ((attended + extra) / (total + extra)) * 100 < required:
          extra += 1
      print("Attendance is below 75%.")
      print("Additional consecutive classes needed:", extra)
      final_percentage = ((attended + extra) / (total + extra)) * 100
   print(f"Final attendance: {final_percentage:.2f}%")
```

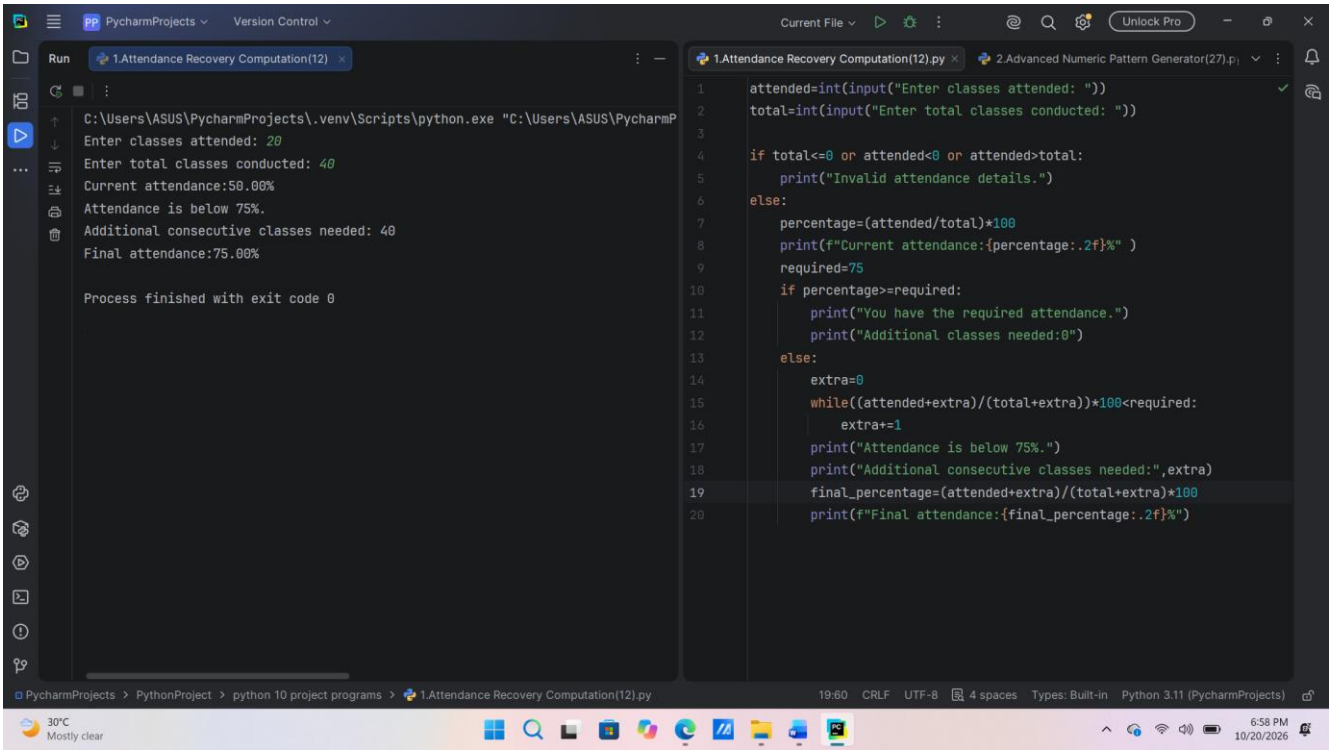
8. Test Cases

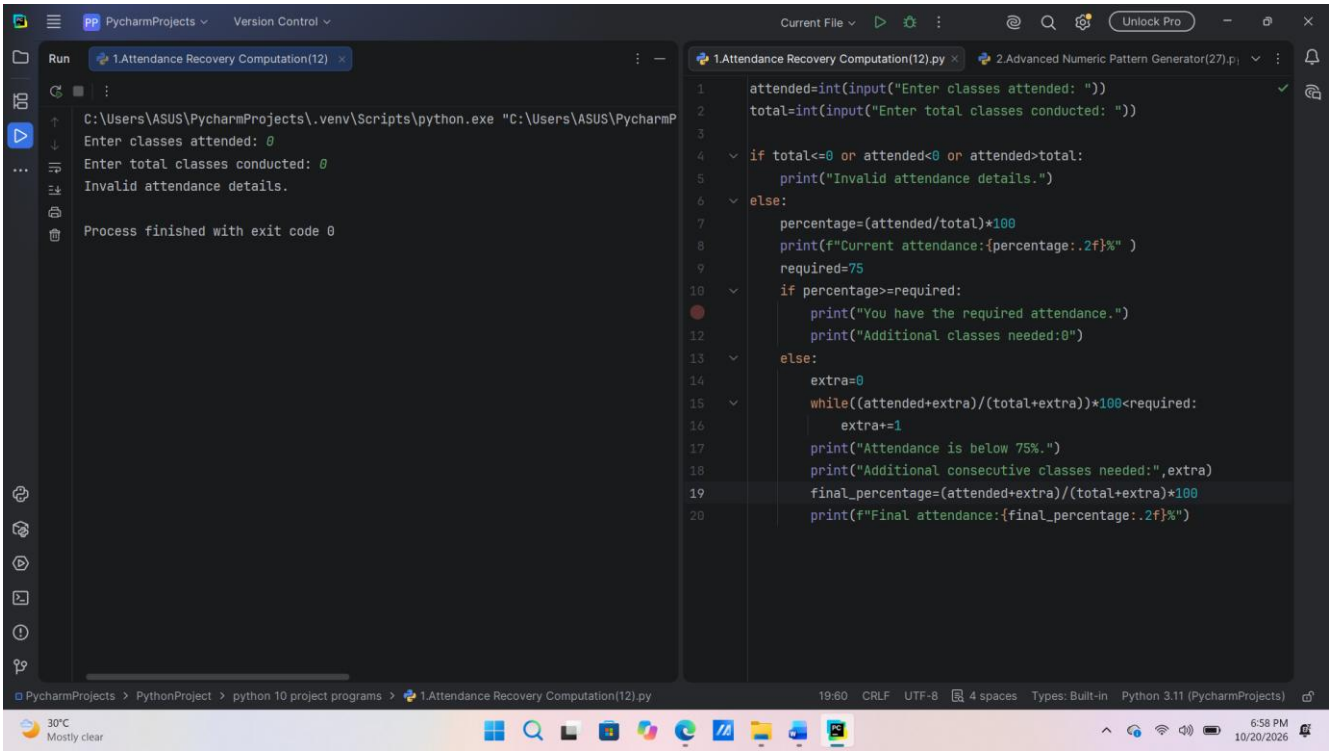
Input	Expected result
Total=40, attended=30	75.00%; 0 extra classes
Total=40, attended=20	50.00%; 40 extra consecutive classes
Total=0, attended=0	Invalid attendance values
Attended=45, total=40	Invalid attendance values

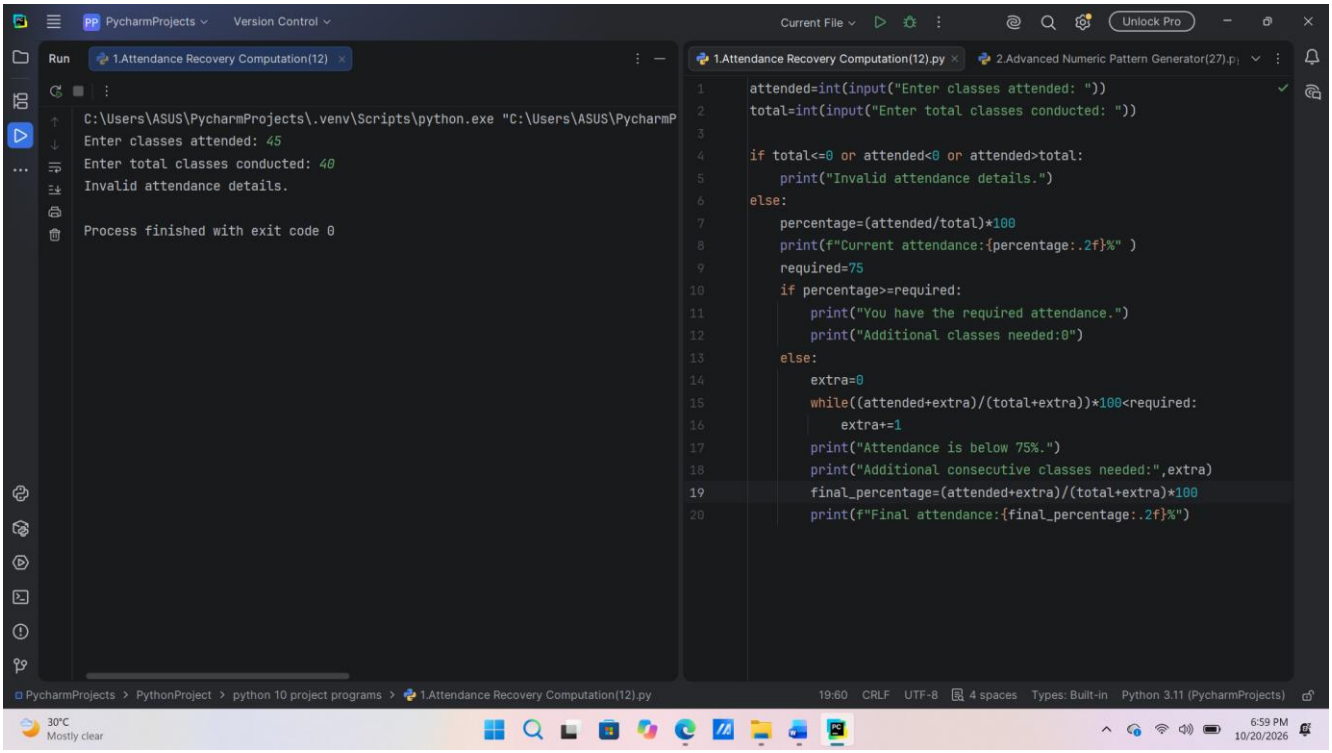
9. Output Screenshots

Sample output:




PycharmProjects
Version Control
Current File
Unlock Pro
Run
1.Attendance Recovery Computation(12)
1.Attendance Recovery Computation(12).py
2.Advanced Numeric Pattern Generator(27).p
C:\Users\ASUS\PycharmProjects\.venv\Scripts\python.exe "C:\Users\ASUS\PycharmP
Enter classes attended: 20
Enter total classes conducted: 40
Current attendance:50.00%
Attendance is below 75%.
Additional consecutive classes needed: 40
Final attendance:75.00%
Process finished with exit code 0
attended=int(input("Enter classes attended: "))
total=int(input("Enter total classes conducted: "))
if total<=0 or attended<0 or attended>total:
print("Invalid attendance details.")
else:
percentage=(attended/total)*100
print(f"Current attendance:{percentage:.2f}%")
required=75
if percentage>=required:
print("You have the required attendance.")
print("Additional classes needed:0")
else:
extra=0
while((attended+extra)/(total+extra))*100<required:
extra+=1
print("Attendance is below 75%.")
print("Additional consecutive classes needed:",extra)
final_percentage=(attended+extra)/(total+extra)*100
print(f"Final attendance:{final_percentage:.2f}%")
PycharmProjects > PythonProject > python 10 project programs > 1.Attendance Recovery Computation(12).py
19:60
CRLF
UTF-8
4 spaces
Types: Built-in
Python 3.11 (PycharmProjects)
30°C
Mostly clear
6:58 PM
10/20/2026


PycharmProjects
Version Control
Current File
Unlock Pro
Run
1.Attendance Recovery Computation(12)
1.Attendance Recovery Computation(12).py
2.Advanced Numeric Pattern Generator(27).p
C:\Users\ASUS\PycharmProjects\.venv\Scripts\python.exe "C:\Users\ASUS\PycharmP
Enter classes attended: 0
Enter total classes conducted: 0
Invalid attendance details.
Process finished with exit code 0
attended=int(input("Enter classes attended: "))
total=int(input("Enter total classes conducted: "))
if total<=0 or attended<0 or attended>total:
print("Invalid attendance details.")
else:
percentage=(attended/total)*100
print(f"Current attendance:{percentage:.2f}%")
required=75
if percentage>=required:
print("You have the required attendance.")
print("Additional classes needed:0")
else:
extra=0
while((attended+extra)/(total+extra))*100<required:
extra+=1
print("Attendance is below 75%.")
print("Additional consecutive classes needed:",extra)
final_percentage=(attended+extra)/(total+extra)*100
print(f"Final attendance:{final_percentage:.2f}%")
PycharmProjects > PythonProject > python 10 project programs > 1.Attendance Recovery Computation(12).py
19:60
CRLF
UTF-8
4 spaces
Types: Built-in
Python 3.11 (PycharmProjects)
30°C
Mostly clear
6:58 PM
10/20/2026



10. Conclusion

The program calculates attendance percentage and finds the minimum number of consecutive classes required to reach 75%, while checking invalid input.

Case Study 2: Advanced Numeric Pattern Generator

Case Study No. 27 | Difficulty: Easy

1. Problem Statement

Write a Python program that accepts a positive odd integer and prints a centered, symmetric numeric diamond. Reject invalid dimensions.

2. Objectives

- Accept a positive integer from the user.
- Use loops to print a triangle pattern.
- Understand nested loops and formatted output.

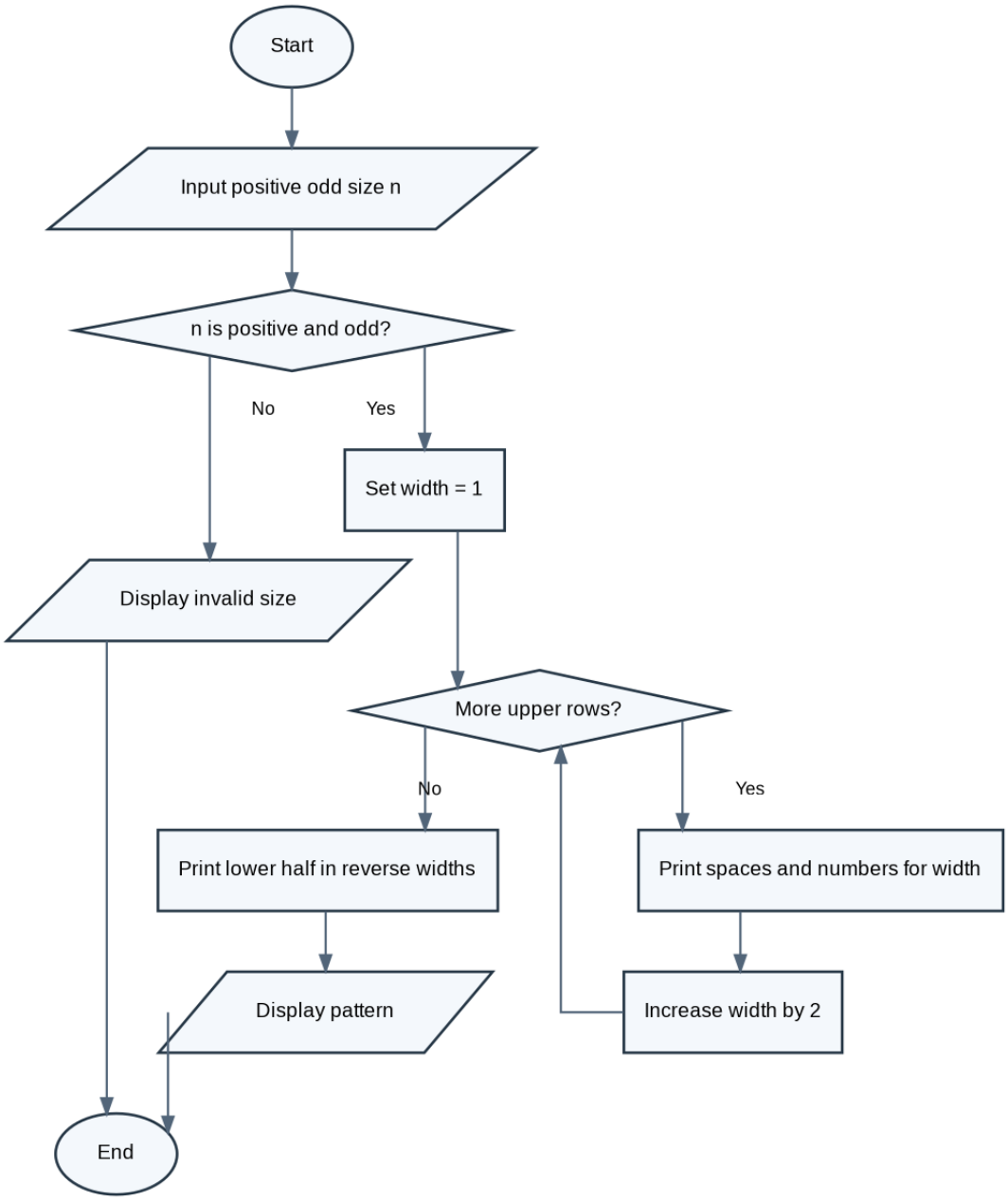
3. Assumptions

- The input n must be a positive integer.
- The pattern begins with 1 and grows by one number on each row.

4. Algorithm

1.Start.
2.Read n.
3.If n is not positive, display an error.
4.For each row from 1 to n, print numbers from 1 to the row number.
5.Move to a new line after each row.
6.Stop.

5. Flowchart



6. Pseudocode

```
START
INPUT n
IF n <= 0 THEN
    PRINT "Enter a positive integer"
ELSE
    FOR row = 1 TO n
        FOR number = 1 TO row
            PRINT number WITHOUT NEWLINE
        END FOR
        PRINT NEWLINE
    END FOR
END IF
STOP
```

7. Python Source Code

```
size = int(input("Enter an odd size: "))

if size <= 0 or size % 2 == 0:
    print("Invalid size. Enter a positive odd number.")
else:
    for width in range(1, size + 1, 2):
        print(" " * ((size - width) // 2), end="")
        for number in range(1, width + 1):
            print(number, end="")
        print()

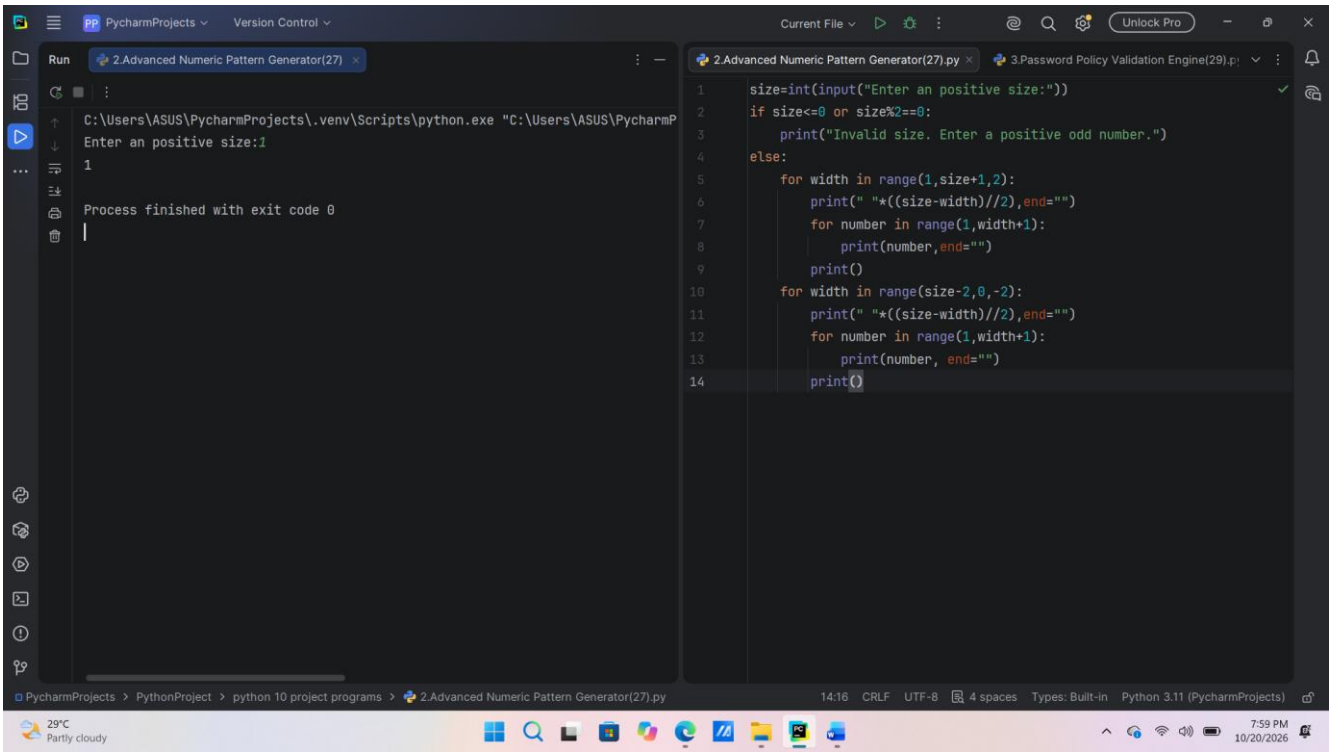
    for width in range(size - 2, 0, -2):
        print(" " * ((size - width) // 2), end="")
        for number in range(1, width + 1):
            print(number, end="")
        print()
```

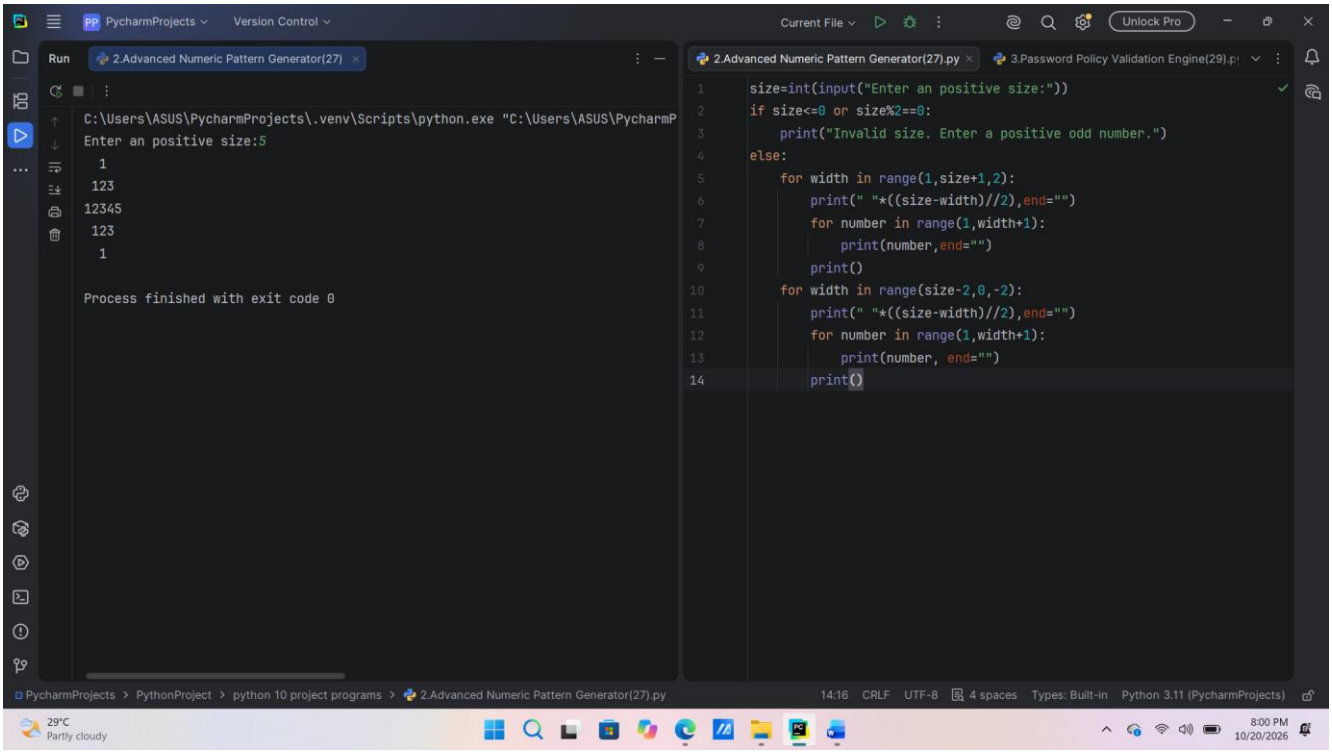
8. Test Cases

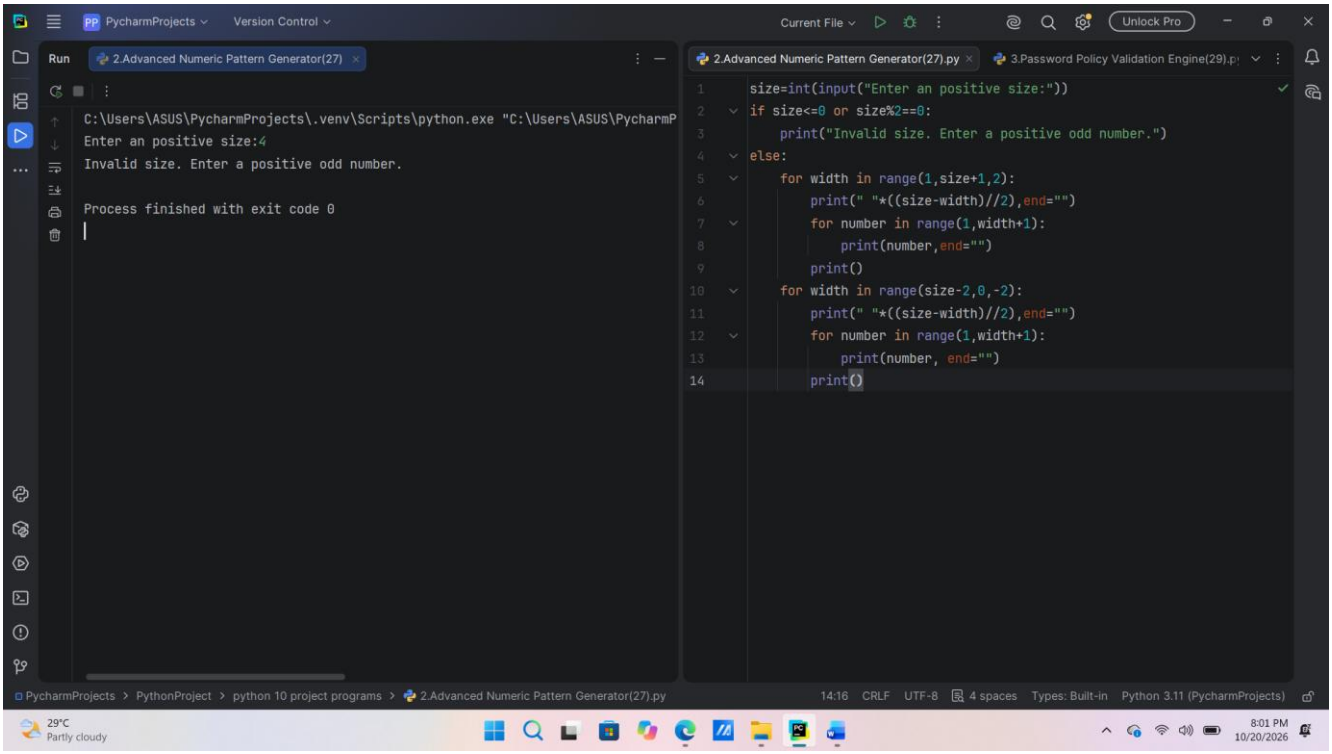
Input	Expected result
Size=1	One row: 1
Size=5	Rows: 1, 123, 12345, 123, 1
Size=4	Invalid; size must be positive and odd
Size=0	Invalid size message

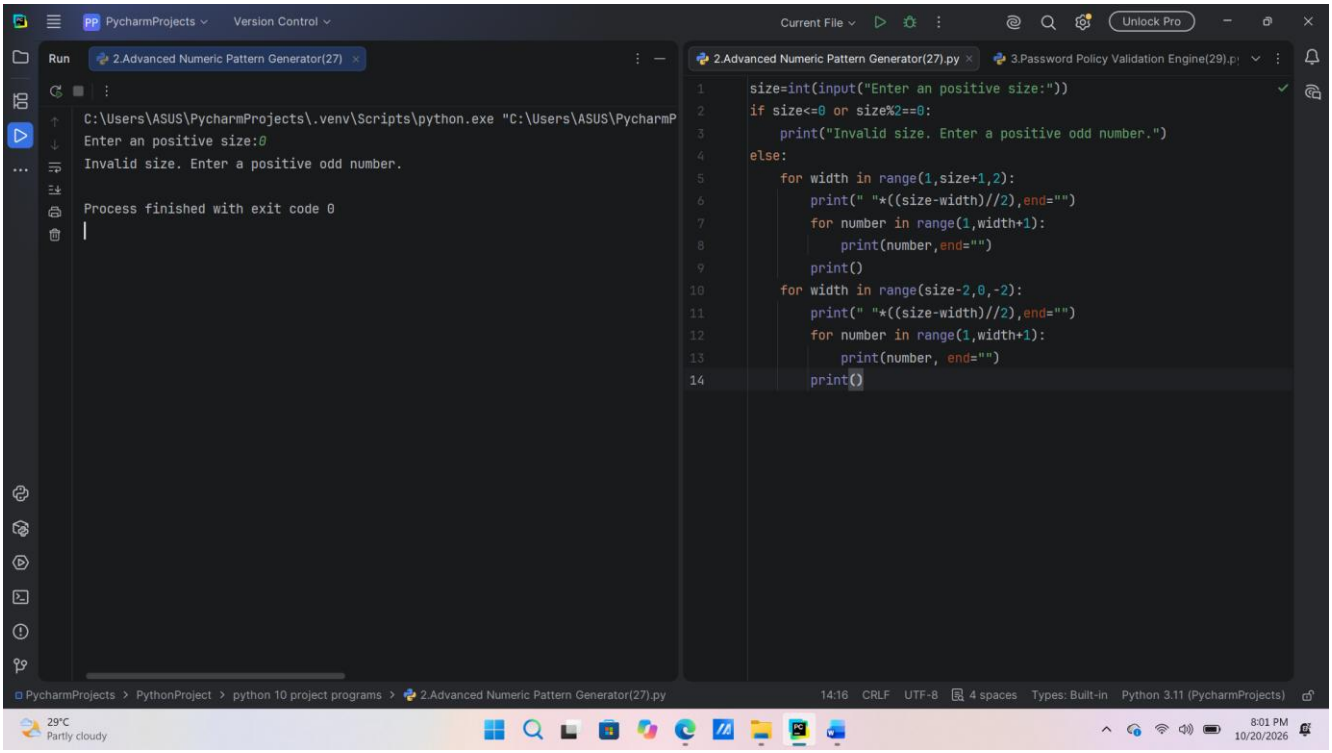
9. Output Screenshots

Sample output :




PycharmProjects
Version Control
Current File
Unlock Pro
Run
2.Advanced Numeric Pattern Generator(27)
C:\Users\ASUS\PycharmProjects\.venv\Scripts\python.exe "C:\Users\ASUS\PycharmP
Enter an positive size:5
1
123
12345
123
1
Process finished with exit code 0
2.Advanced Numeric Pattern Generator(27).py
3.Password Policy Validation Engine(29).p
size=int(input("Enter an positive size:"))
if size<=0 or size%2==0:
print("Invalid size. Enter a positive odd number.")
else:
for width in range(1,size+1,2):
print(" "*((size-width)//2),end="")
for number in range(1,width+1):
print(number,end="")
print()
for width in range(size-2,0,-2):
print(" "*((size-width)//2),end="")
for number in range(1,width+1):
print(number, end="")
print()
PycharmProjects > PythonProject > python 10 project programs > 2.Advanced Numeric Pattern Generator(27).py
14:16 CRLF UTF-8 4 spaces Types: Built-in Python 3.11 (PycharmProjects)
29°C
Partly cloudy
8:00 PM
10/20/2026


PycharmProjects
Version Control
Current File
Unlock Pro
Run
2.Advanced Numeric Pattern Generator(27)
C:\Users\ASUS\PycharmProjects\.venv\Scripts\python.exe "C:\Users\ASUS\PycharmP
Enter an positive size:4
Invalid size. Enter a positive odd number.
Process finished with exit code 0
2.Advanced Numeric Pattern Generator(27).py
3.Password Policy Validation Engine(29).p
size=int(input("Enter an positive size:"))
if size<=0 or size%2==0:
print("Invalid size. Enter a positive odd number.")
else:
for width in range(1,size+1,2):
print(" "*((size-width)//2),end="")
for number in range(1,width+1):
print(number,end="")
print()
for width in range(size-2,0,-2):
print(" "*((size-width)//2),end="")
for number in range(1,width+1):
print(number, end="")
print()
PycharmProjects > PythonProject > python 10 project programs > 2.Advanced Numeric Pattern Generator(27).py
14:16 CRLF UTF-8 4 spaces Types: Built-in Python 3.11 (PycharmProjects)
29°C
Partly cloudy
8:01 PM
10/20/2026



10. Conclusion

The program generates a numeric triangle using nested for loops and demonstrates repetition and row-by-row output.

Case Study 3: Password Policy Validation Engine

Case Study No. 29 | Difficulty: Easy

1. Problem Statement

Create a program that checks whether a password meets a basic policy: at least 8 characters, at least one uppercase letter, one lowercase letter, one digit, and one special character.

2. Objectives

- Accept a password.
- Check each policy requirement independently.
- Report whether it is valid and list missing requirements.

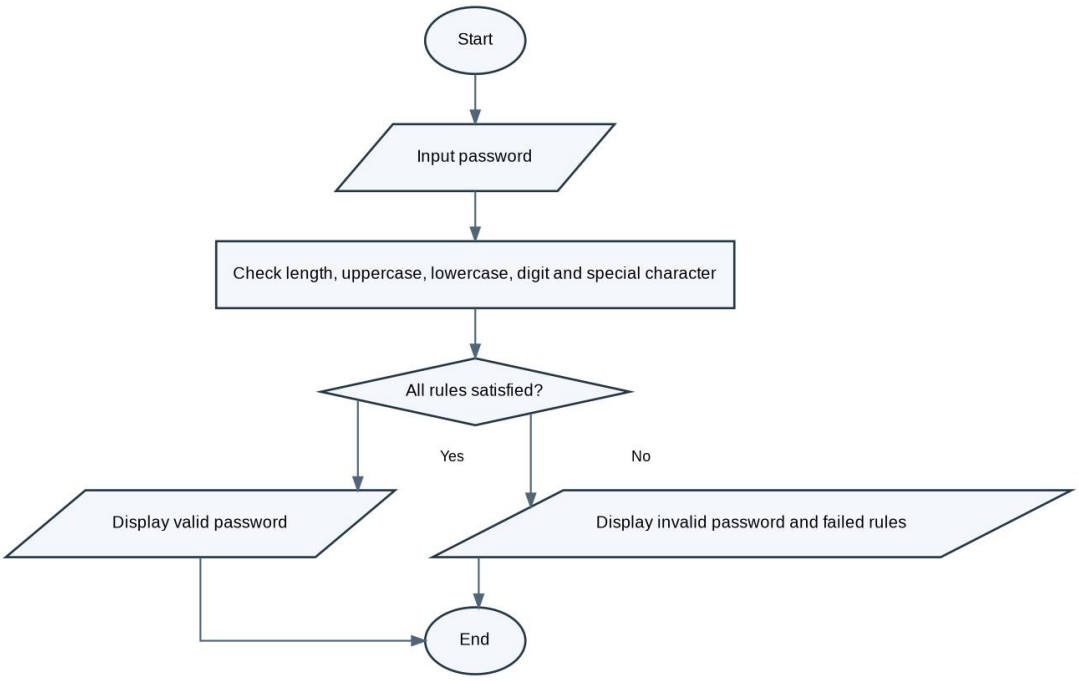
3. Assumptions

- A special character is any non-letter and non-digit character.
- The password is evaluated locally and is not stored.

4. Algorithm

1.Start.
2.Read password.
3.Check its length.
4.Check for uppercase, lowercase, digit, and special character.
5.Collect all failed rules.
6.If no rule fails, display valid; otherwise display invalid and failed rules.
7.Stop.

5. Flowchart



6. Pseudocode

```
START
INPUT password
missing = empty list
IF length(password) < 8: add length rule to missing
IF no uppercase: add uppercase rule
```

```
IF no lowercase: add lowercase rule
IF no digit: add digit rule
IF no special character: add special rule
IF missing is empty: PRINT "Valid password"
ELSE: PRINT "Invalid password" and missing rules
STOP
```

7. Python Source Code

```
password = input("Enter a password to check: ")

length_ok = len(password) >= 8
has_upper = False
has_lower = False
has_digit = False
has_special = False

for ch in password:
    if ch.isupper():
        has_upper = True
    elif ch.islower():
        has_lower = True
    elif ch.isdigit():
        has_digit = True
    else:
        has_special = True

print("At least 8 characters:", length_ok)
print("Contains uppercase:", has_upper)
print("Contains lowercase:", has_lower)
print("Contains a digit:", has_digit)
print("Contains a special character:", has_special)

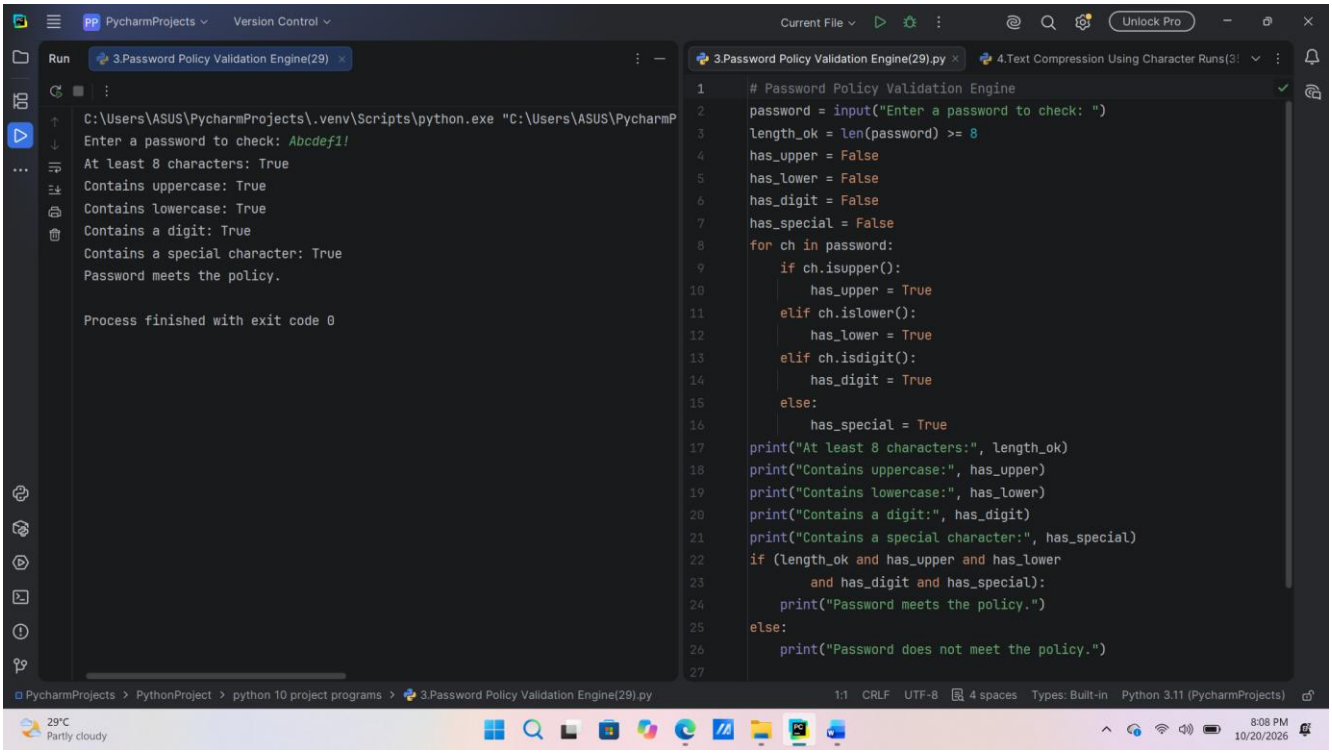
if length_ok and has_upper and has_lower and has_digit and has_special:
    print("Password meets the policy.")
else:
    print("Password does not meet the policy.")
```

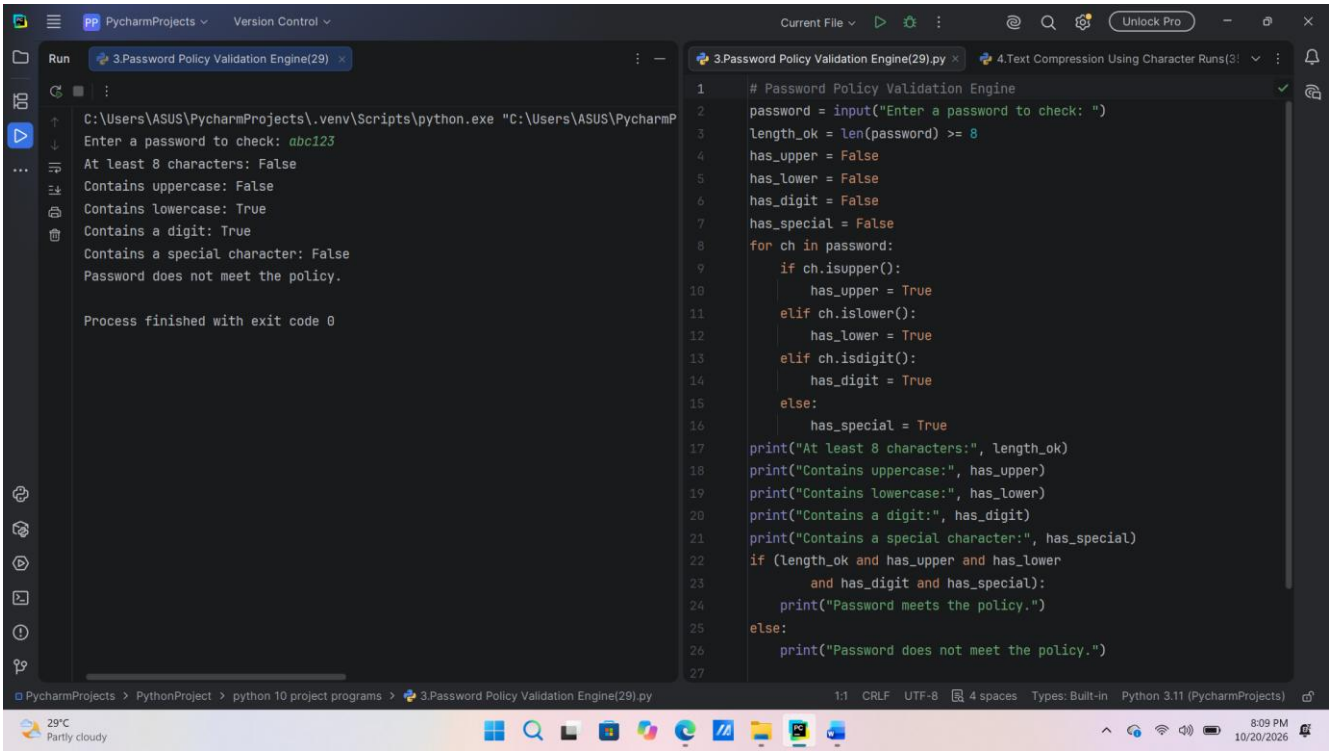
8. Test Cases

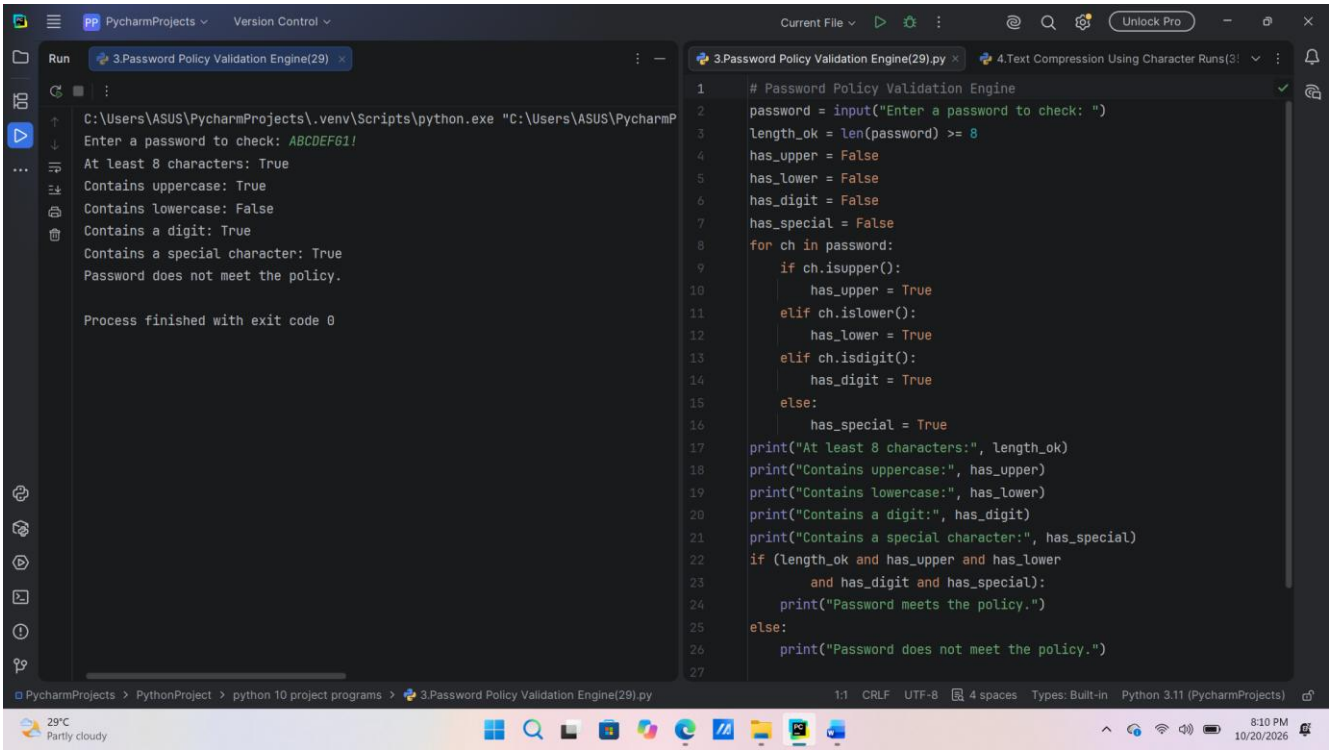
Password	Expected result
Abcdef1!	Valid; all policy rules satisfied
abc123	Invalid; too short, uppercase and special character missing
ABCDEFG1!	Invalid; lowercase missing
Abcdefgh	Invalid; digit and special character missing

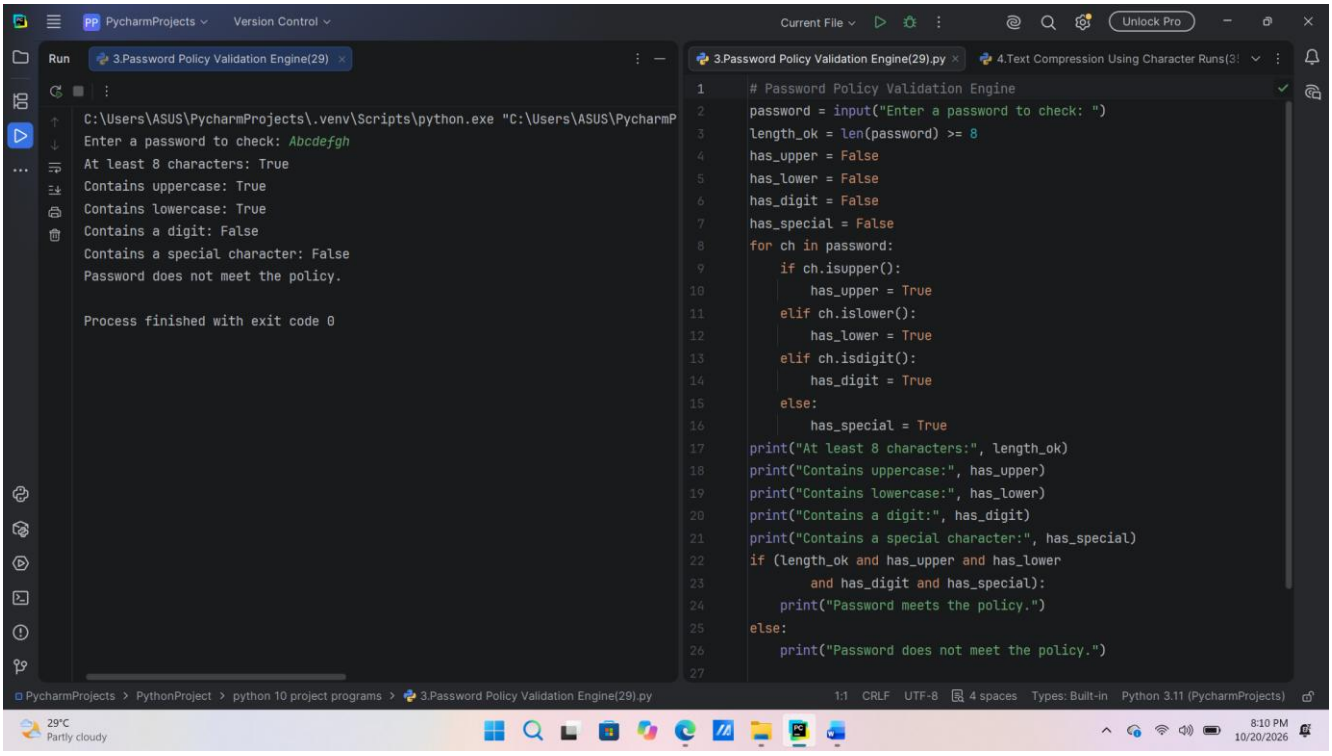
9. Output Screenshots

Sample output :


PycharmProjects
Version Control
Current File
Unlock Pro
Run
3.Password Policy Validation Engine(29)
C:\Users\ASUS\PycharmProjects\.venv\Scripts\python.exe "C:\Users\ASUS\PycharmP
Enter a password to check: Abcdef1!
At least 8 characters: True
Contains uppercase: True
Contains lowercase: True
Contains a digit: True
Contains a special character: True
Password meets the policy.
Process finished with exit code 0
3.Password Policy Validation Engine(29).py
4.Text Compression Using Character Runs(3
Password Policy Validation Engine
password = input("Enter a password to check: ")
length_ok = len(password) >= 8
has_upper = False
has_lower = False
has_digit = False
has_special = False
for ch in password:
if ch.isupper():
has_upper = True
elif ch.islower():
has_lower = True
elif ch.isdigit():
has_digit = True
else:
has_special = True
print("At least 8 characters:", length_ok)
print("Contains uppercase:", has_upper)
print("Contains lowercase:", has_lower)
print("Contains a digit:", has_digit)
print("Contains a special character:", has_special)
if (length_ok and has_upper and has_lower
and has_digit and has_special):
print("Password meets the policy.")
else:
print("Password does not meet the policy.")
PycharmProjects > PythonProject > python 10 project programs > 3.Password Policy Validation Engine(29).py
1:1 CRLF UTF-8 4 spaces Types: Built-in Python 3.11 (PycharmProjects)
29°C
Partly cloudy
8:08 PM
10/20/2026


PycharmProjects
Version Control
Current File
Unlock Pro
Run
3.Password Policy Validation Engine(29)
C:\Users\ASUS\PycharmProjects\.venv\Scripts\python.exe "C:\Users\ASUS\PycharmP
Enter a password to check: abc123
At least 8 characters: False
Contains uppercase: False
Contains lowercase: True
Contains a digit: True
Contains a special character: False
Password does not meet the policy.
Process finished with exit code 0
3.Password Policy Validation Engine(29).py
4.Text Compression Using Character Runs(3
Password Policy Validation Engine
password = input("Enter a password to check: ")
length_ok = len(password) >= 8
has_upper = False
has_lower = False
has_digit = False
has_special = False
for ch in password:
if ch.isupper():
has_upper = True
elif ch.islower():
has_lower = True
elif ch.isdigit():
has_digit = True
else:
has_special = True
print("At least 8 characters:", length_ok)
print("Contains uppercase:", has_upper)
print("Contains lowercase:", has_lower)
print("Contains a digit:", has_digit)
print("Contains a special character:", has_special)
if (length_ok and has_upper and has_lower
and has_digit and has_special):
print("Password meets the policy.")
else:
print("Password does not meet the policy.")
PycharmProjects > PythonProject > python 10 project programs > 3.Password Policy Validation Engine(29).py
1:1 CRLF UTF-8 4 spaces Types: Built-in Python 3.11 (PycharmProjects)
29°C
Partly cloudy
8:09 PM
10/20/2026





10. Conclusion

The program validates a password against multiple rules and clearly reports which requirements need to be satisfied.

Case Study 4: Text Compression Using Character Runs

Case Study No. 35 | Difficulty: Easy

1. Problem Statement

Write a program that compresses a string by replacing each consecutive run of identical characters with the character followed by the number of repetitions. For example, “aaabbc” becomes “a3b2c1”.

2. Objectives

- Read a string.
- Identify consecutive repeated characters.
- Create a compressed representation using run lengths.
- Handle an empty string safely.

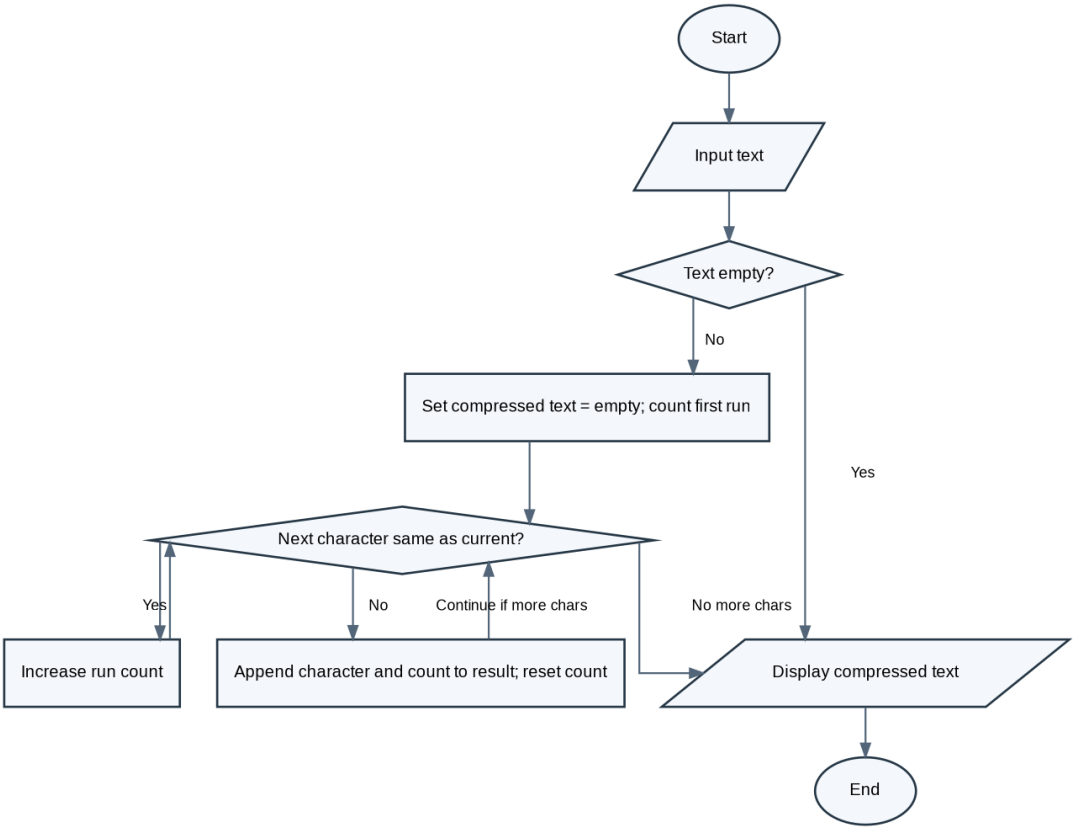
3. Assumptions

- Characters are case-sensitive.
- The program performs simple run-length encoding; it does not guarantee the compressed string is shorter.

4. Algorithm

1.Start.
2.Read text.
3.If text is empty, display an empty result.
4.Set current character and count.
5.Scan the remaining characters.
6.When character changes, append previous character and count, then reset count.
7.Append the final run.
8.Display compressed text.
9.Stop.

5. Flowchart



6. Pseudocode

```
START
INPUT text
IF text is empty THEN
    PRINT empty result
ELSE
    result = ""
    current = first character
    count = 1
    FOR each next character ch
        IF ch == current THEN count = count + 1
        ELSE append current and count to result
             current = ch; count = 1
        END IF
    END FOR
    append current and count to result
    PRINT result
END IF
STOP
```

7. Python Source Code

```
text = input("Enter text to compress: ")

if text == "":
    print("Compressed text:")
else:
    result = ""
    current = text[0]
    count = 1
```

```
for ch in text[1:]:
    if ch == current:
        count += 1
    else:
        result += current + str(count)
        current = ch
        count = 1

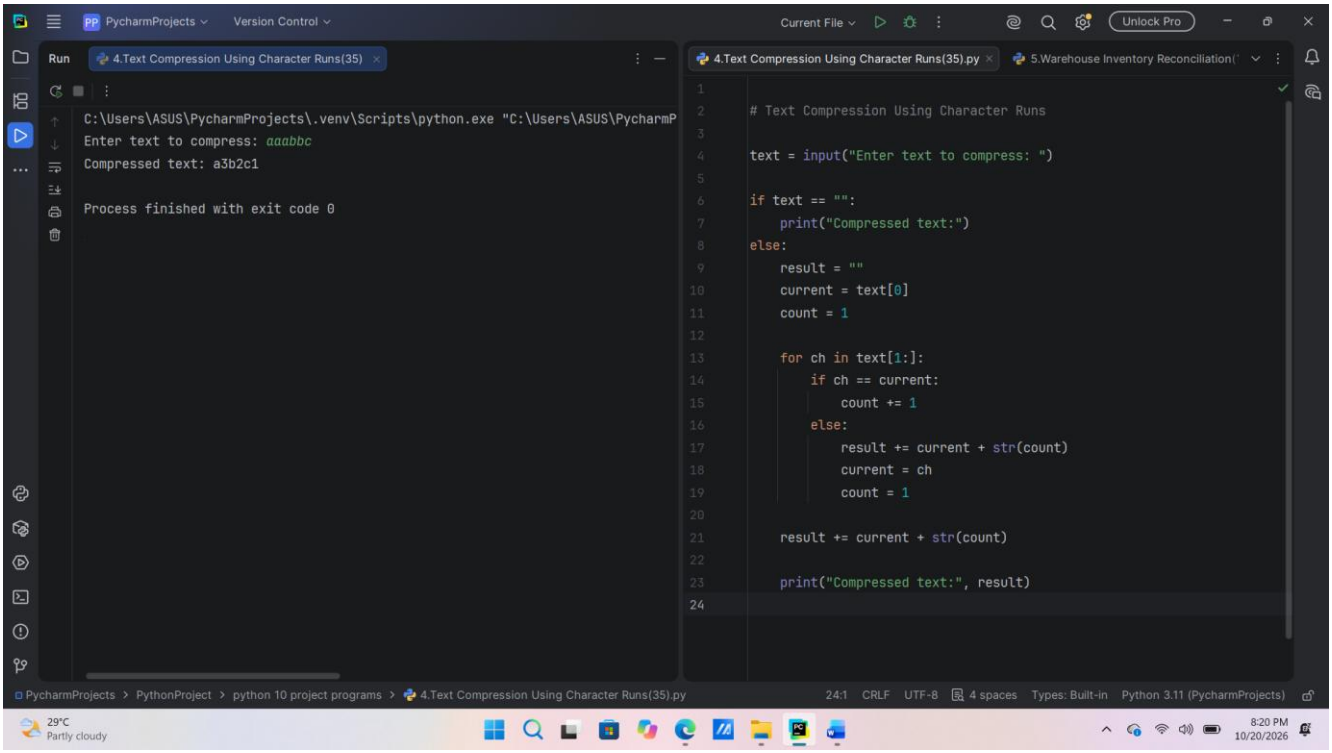
result += current + str(count)
print("Compressed text:", result)
```

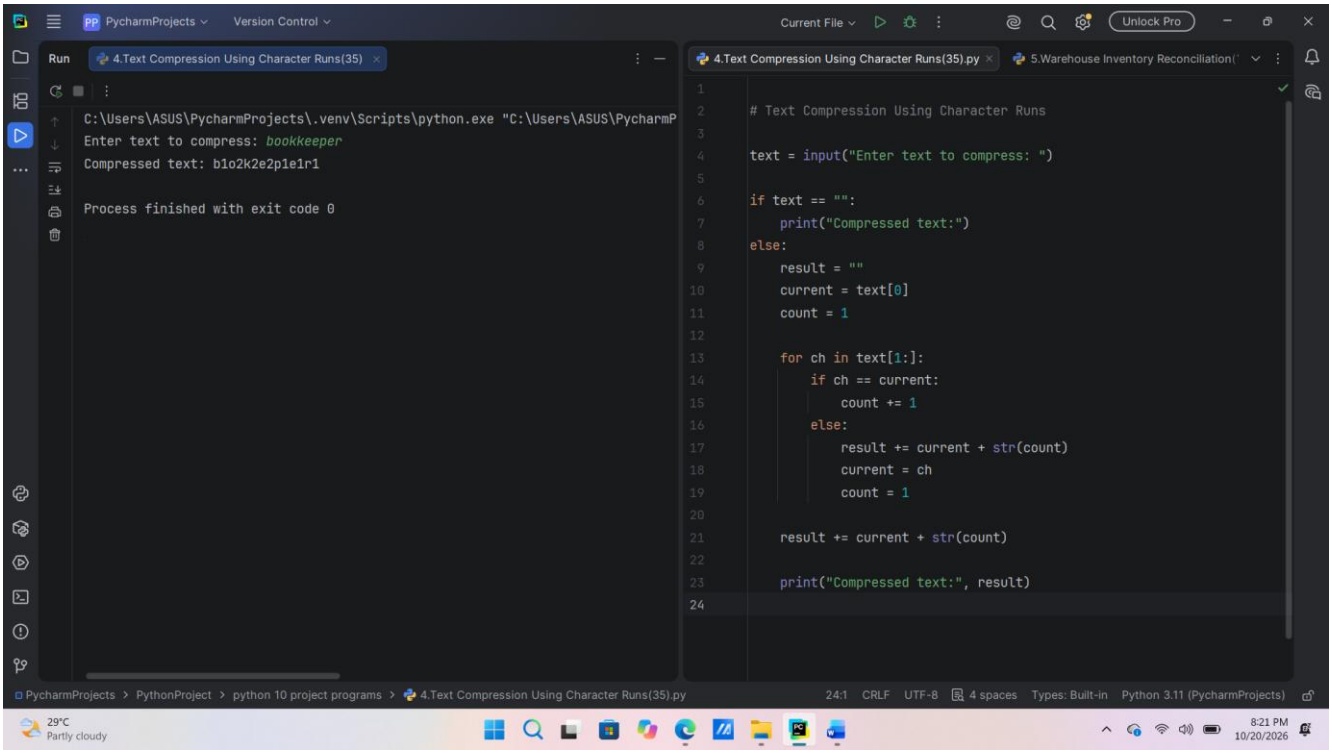
8. Test Cases

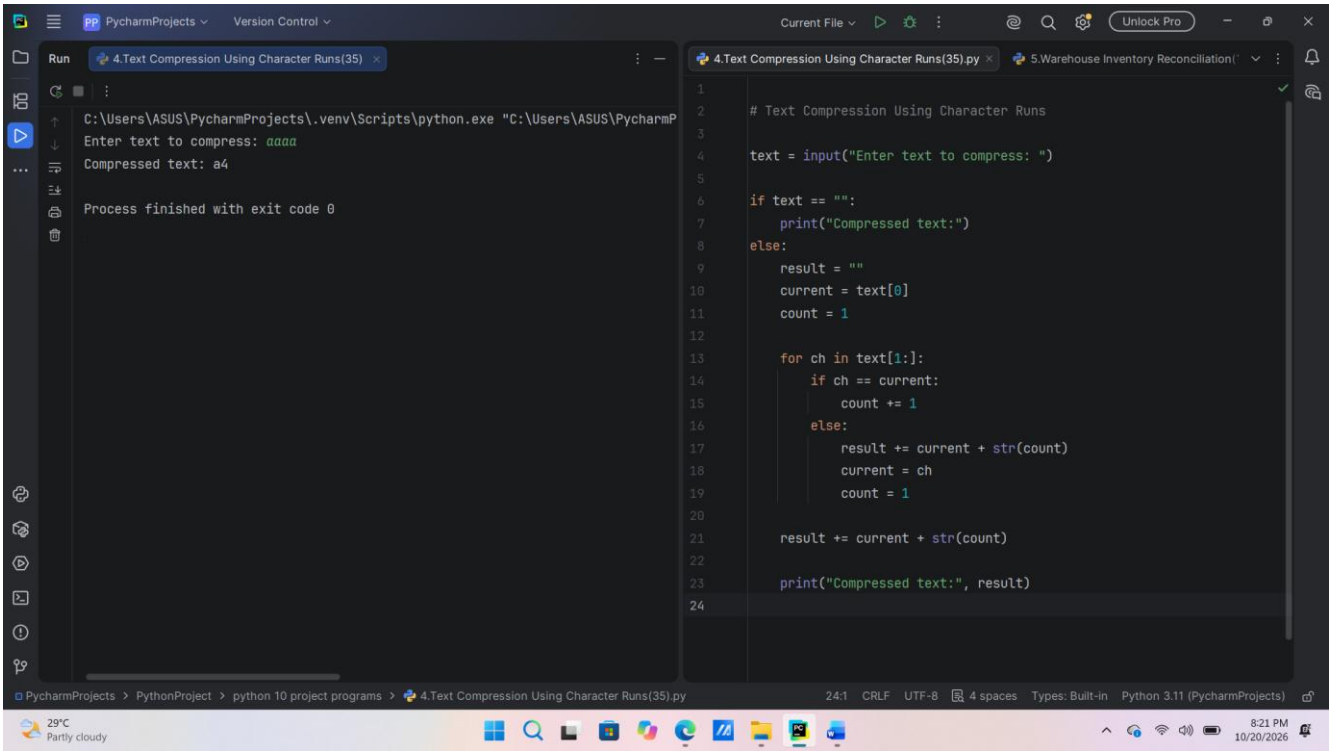
Input text	Expected result
aaabbc	a3b2c1
bookkeeper	b1o2k2e2p1e1r1
aaaa	a4
Empty string	Empty result (depending on implementation)

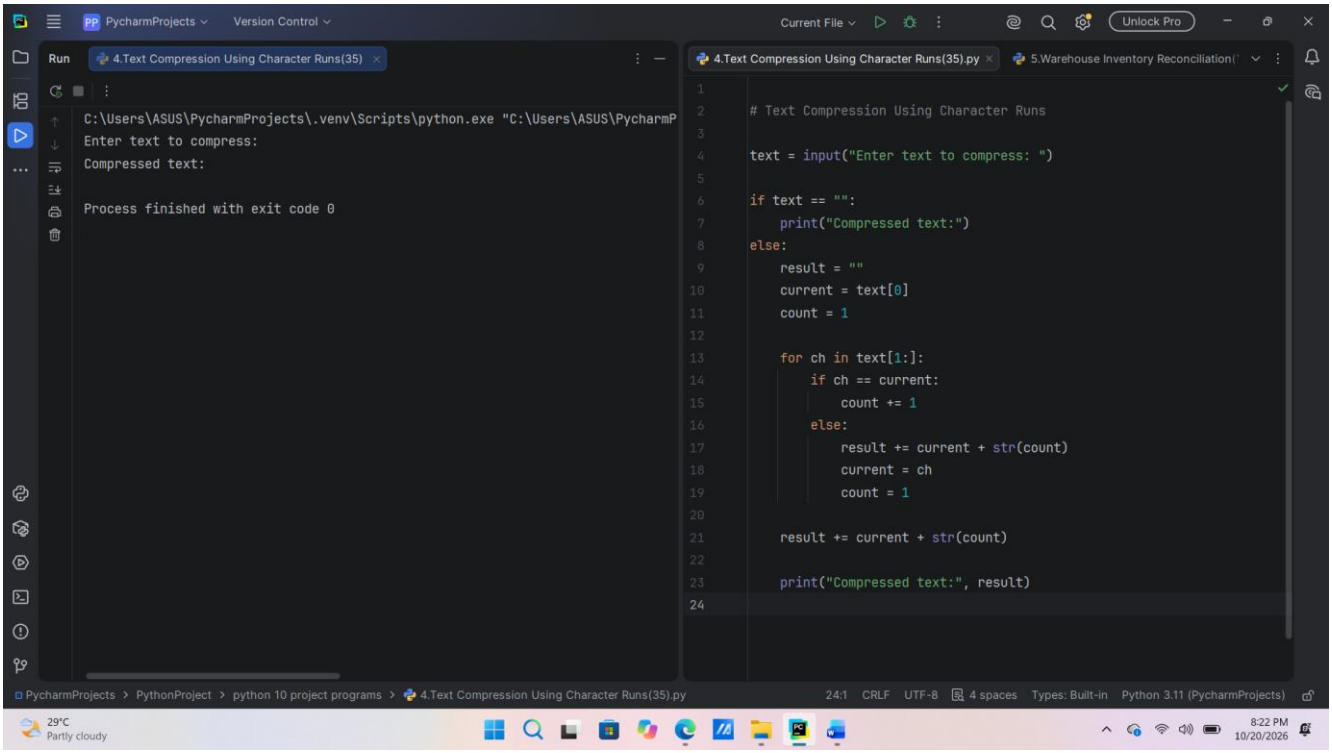
9. Output Screenshots

Sample console output :




PycharmProjects
Version Control
Current File
Unlock Pro
Run
4.Text Compression Using Character Runs(35)
C:\Users\ASUS\PycharmProjects\.venv\Scripts\python.exe "C:\Users\ASUS\PycharmP
Enter text to compress: bookkeeper
Compressed text: b1o2k2e2p1e1r1
Process finished with exit code 0
4.Text Compression Using Character Runs(35).py
5.Warehouse Inventory Reconciliation(
Text Compression Using Character Runs
text = input("Enter text to compress: ")
if text == "":
print("Compressed text:")
else:
result = ""
current = text[0]
count = 1
for ch in text[1:]:
if ch == current:
count += 1
else:
result += current + str(count)
current = ch
count = 1
result += current + str(count)
print("Compressed text:", result)
PycharmProjects > PythonProject > python 10 project programs > 4.Text Compression Using Character Runs(35).py
24:1 CRLF UTF-8 4 spaces Types: Built-in Python 3.11 (PycharmProjects)
29°C
Partly cloudy
8:21 PM
10/20/2026


PycharmProjects
Version Control
Current File
Unlock Pro
Run
4.Text Compression Using Character Runs(35)
C:\Users\ASUS\PycharmProjects\.venv\Scripts\python.exe "C:\Users\ASUS\PycharmP
Enter text to compress: aaaa
Compressed text: a4
Process finished with exit code 0
4.Text Compression Using Character Runs(35).py
5.Warehouse Inventory Reconciliation(
Text Compression Using Character Runs
text = input("Enter text to compress: ")
if text == "":
print("Compressed text:")
else:
result = ""
current = text[0]
count = 1
for ch in text[1:]:
if ch == current:
count += 1
else:
result += current + str(count)
current = ch
count = 1
result += current + str(count)
print("Compressed text:", result)
PycharmProjects > PythonProject > python 10 project programs > 4.Text Compression Using Character Runs(35).py
24:1 CRLF UTF-8 4 spaces Types: Built-in Python 3.11 (PycharmProjects)
29°C
Partly cloudy
8:21 PM
10/20/2026



10. Conclusion

The program applies run-length encoding by counting adjacent repeated characters and building a compressed string.

Case Study 5: Warehouse Inventory Reconciliation

Case Study No. 14 | Difficulty: Medium

1. Problem Statement

Build a program that compares recorded stock with physically counted stock for several items and reports each item's difference and whether it has a shortage, surplus, or matching quantity.

2. Objectives

- Store item names and recorded quantities.
- Accept physical counts.
- Calculate physical minus recorded quantity.
- Classify every item and show a summary.

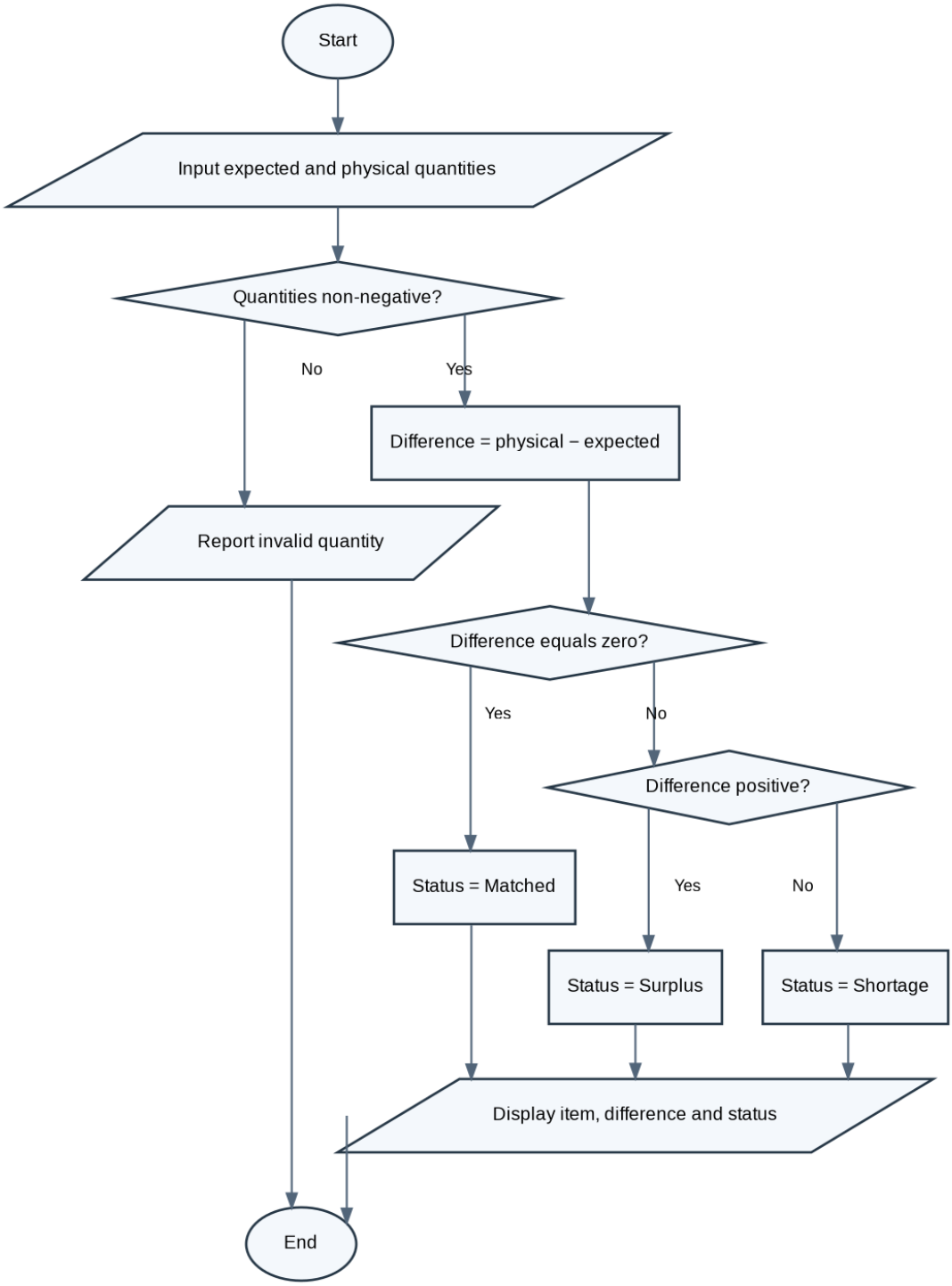
3. Assumptions

- Quantities are non-negative integers.
- A difference below zero means shortage; above zero means surplus; zero means matched.
- Item names are unique for this exercise.

4. Algorithm

1.Start.
2.Input number of items.
3.For each item, read name, recorded stock, and physical stock.
4.Calculate difference = physical − recorded.
5.Classify as shortage, surplus, or matched.
6.Display the report for each item.
7.Stop.

5. Flowchart



6. Pseudocode

```
START
INPUT item_count
REPEAT item_count times
    INPUT item_name, recorded, physical
    difference = physical - recorded
    IF difference < 0: status = "Shortage"
    ELSE IF difference > 0: status = "Surplus"
    ELSE: status = "Matched"
    PRINT item_name, recorded, physical, difference, status
```

```
END REPEAT
STOP
```

7. Python Source Code

```
n = int(input("Enter number of inventory items: "))

if n <= 0:
    print("Enter a positive number of items.")
else:
    shortage = 0
    surplus = 0
    matched = 0

    for i in range(1, n + 1):
        print("\nItem", i)
        name = input("Enter item name: ")
        expected = int(input("Enter expected quantity: "))
        physical = int(input("Enter physical quantity: "))

        if expected < 0 or physical < 0:
            print("Quantities cannot be negative. Item skipped.")
            continue

        difference = physical - expected
        if difference < 0:
            status = "Shortage"
            shortage += 1
        elif difference > 0:
            status = "Surplus"
            surplus += 1
        else:
            status = "Matched"
            matched += 1

        print("Item:", name)
        print("Difference:", difference)
        print("Status:", status)

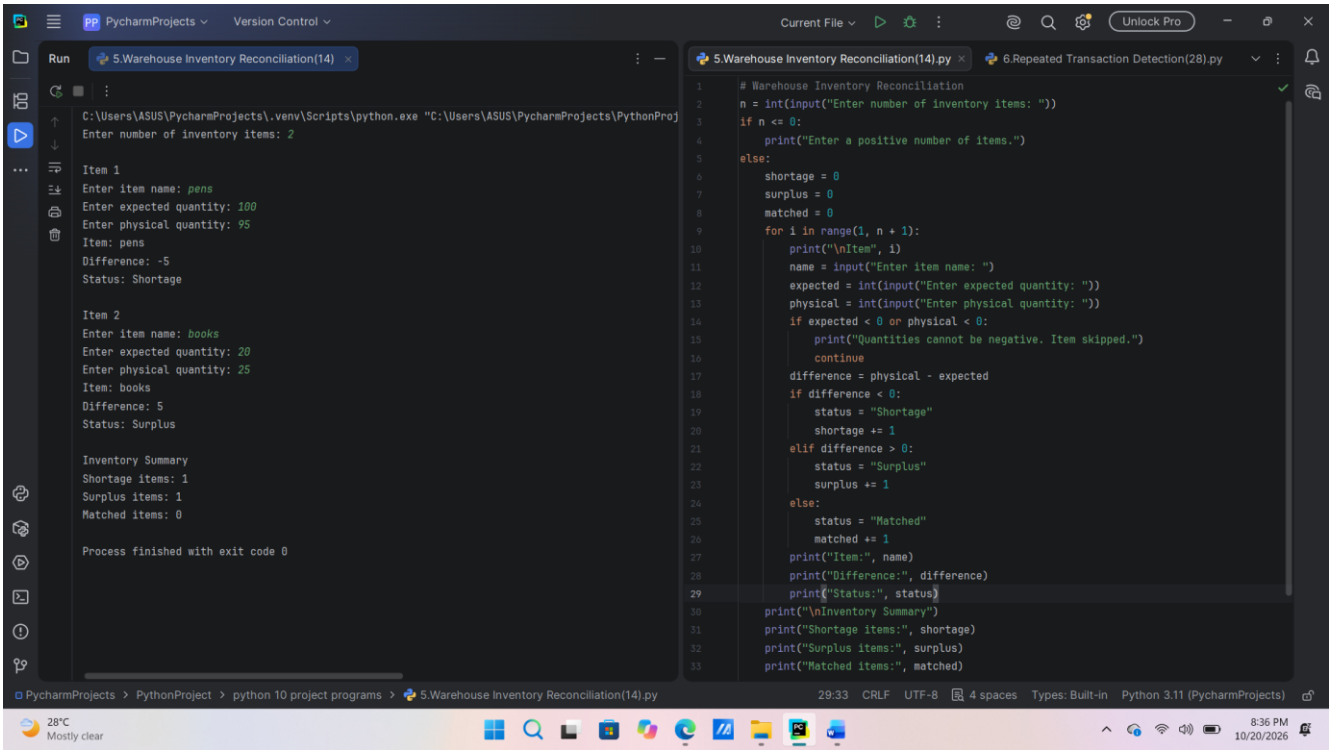
    print("\nInventory Summary")
    print("Shortage items:", shortage)
    print("Surplus items:", surplus)
    print("Matched items:", matched)
```

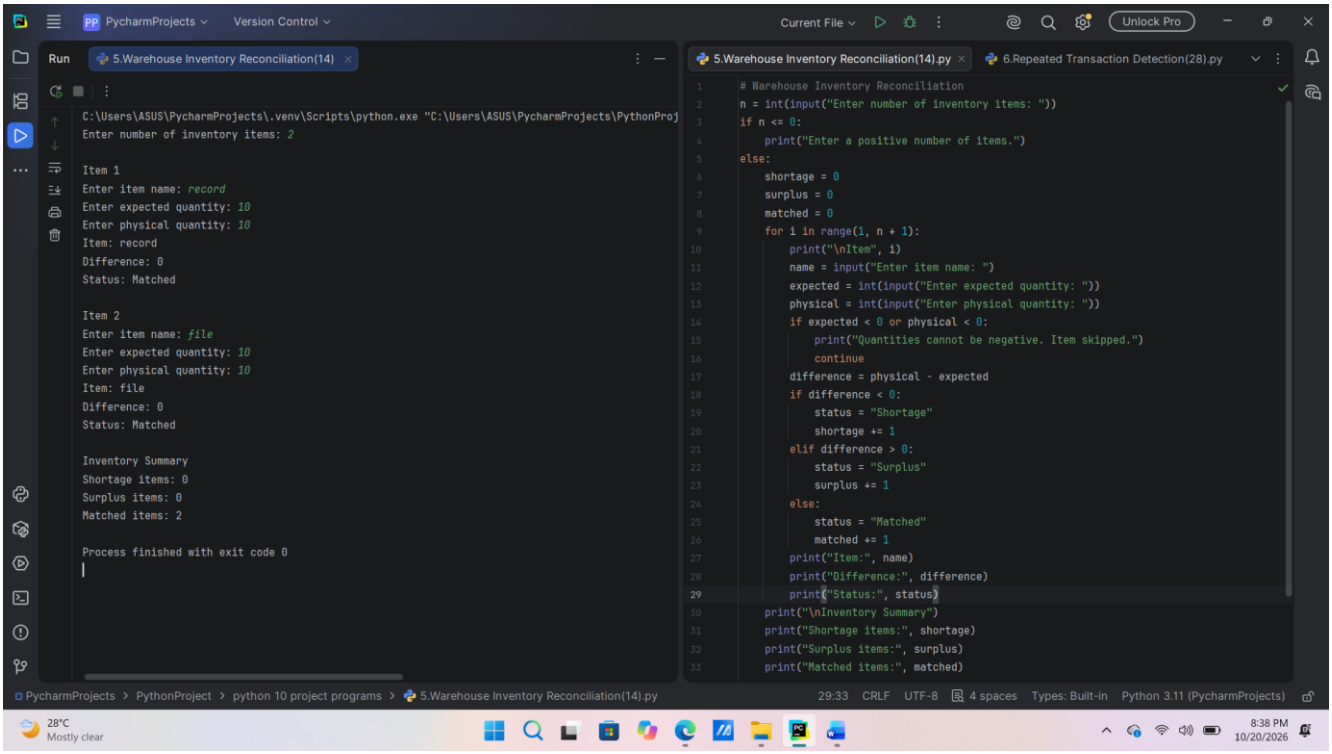
8. Test Cases

Expected / physical	Expected result
100 / 95	Shortage of 5
20 / 25	Surplus of 5
10 / 10	Matched; difference 0
-1 / 5	Invalid quantity

9. Output Screenshots

Sample output :





10. Conclusion

The program reconciles recorded and physical inventory, calculates discrepancies, and labels each item for follow-up.

Case Study 6: Repeated Transaction Detection

Case Study No. 28 | Difficulty: Medium

1. Problem Statement

Create a program that reads transaction IDs and identifies repeated IDs so that duplicate transactions can be reviewed.

2. Objectives

- Accept a list of transaction IDs.
- Detect IDs that occur more than once.
- Display duplicate IDs and their counts.

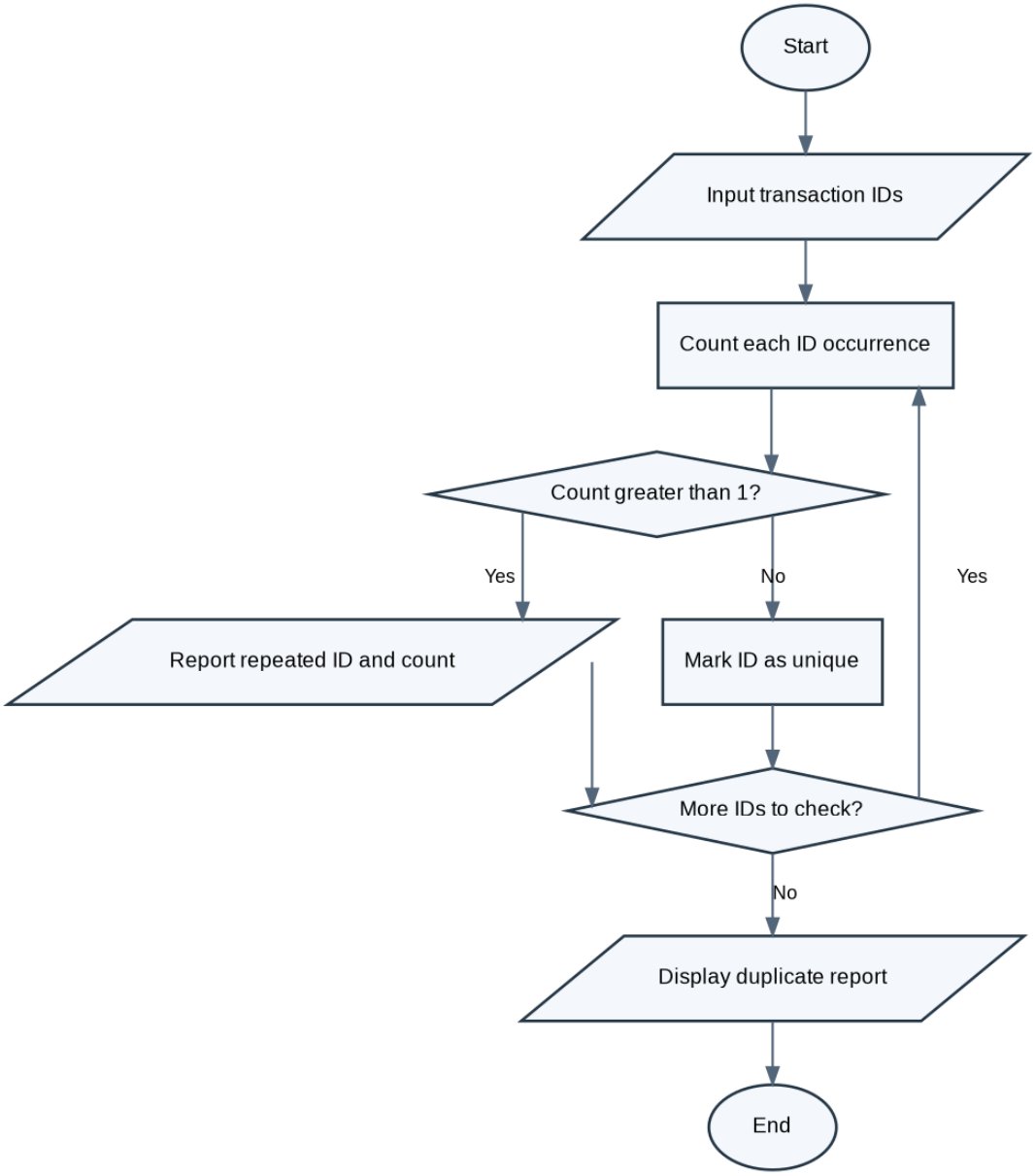
3. Assumptions

- Transaction IDs are compared as strings.
- Repeated occurrences are counted; the same ID is treated as a duplicate after its first occurrence.

4. Algorithm

1.Start.
2.Read the number of transactions.
3.Read all transaction IDs into a list.
4.Count each ID using a dictionary.
5.Select IDs with count greater than one.
6.Display duplicates and counts or report none.
7.Stop.

5. Flowchart



6. Pseudocode

```
START
INPUT n
INPUT n transaction IDs
counts = empty dictionary
FOR each ID
    increase counts[ID]
END FOR
found = false
FOR each ID and count in counts
    IF count > 1: PRINT ID and count; found = true
END FOR
IF found == false: PRINT "No duplicates"
STOP
```

7. Python Source Code

```
n = int(input("Enter number of transactions: "))
```

```
if n <= 0:
    print("Enter a positive number of transactions.")
else:
    ids = []
    for i in range(n):
        transaction_id = input("Enter transaction ID: ").strip()
        ids.append(transaction_id)

    duplicates_found = False
    reported = []
    for transaction_id in ids:
        count = ids.count(transaction_id)
        if count > 1 and transaction_id not in reported:
            print("Repeated ID:", transaction_id, "| Count:", count)
            reported.append(transaction_id)
            duplicates_found = True

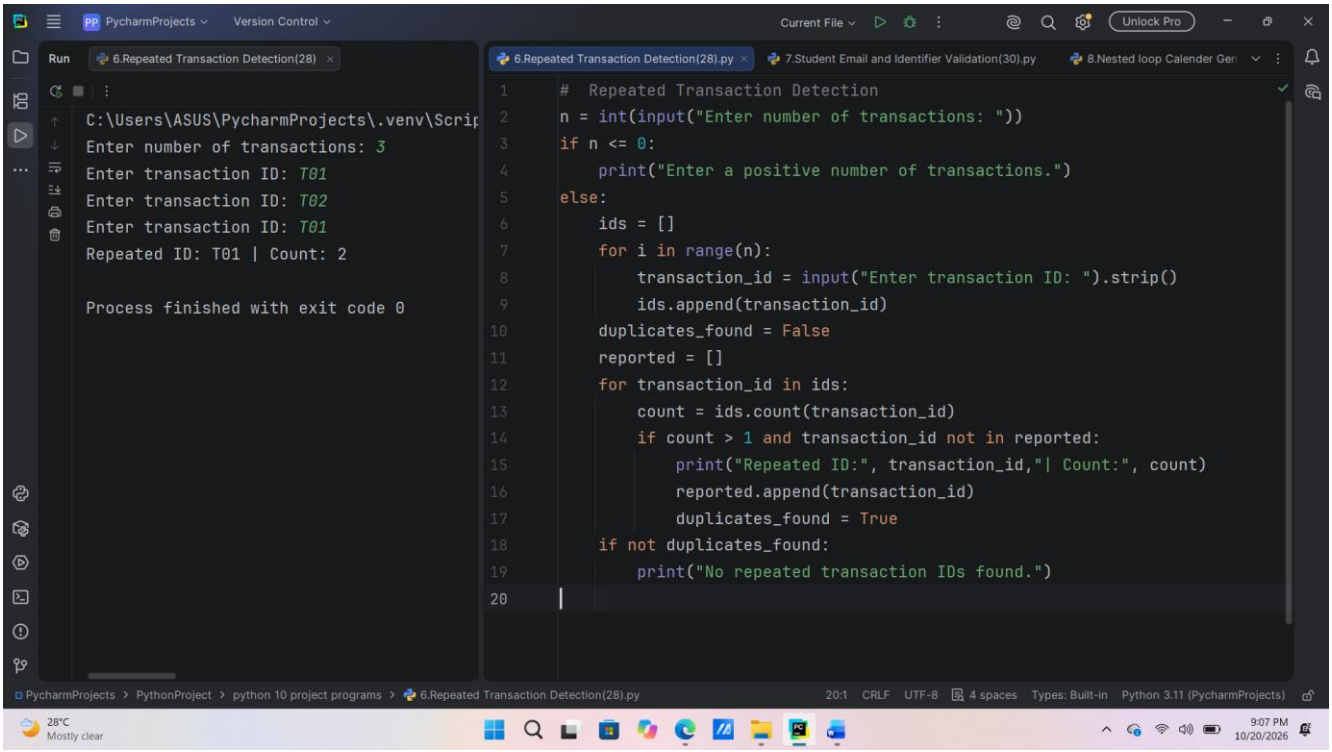
    if not duplicates_found:
        print("No repeated transaction IDs found.")
```

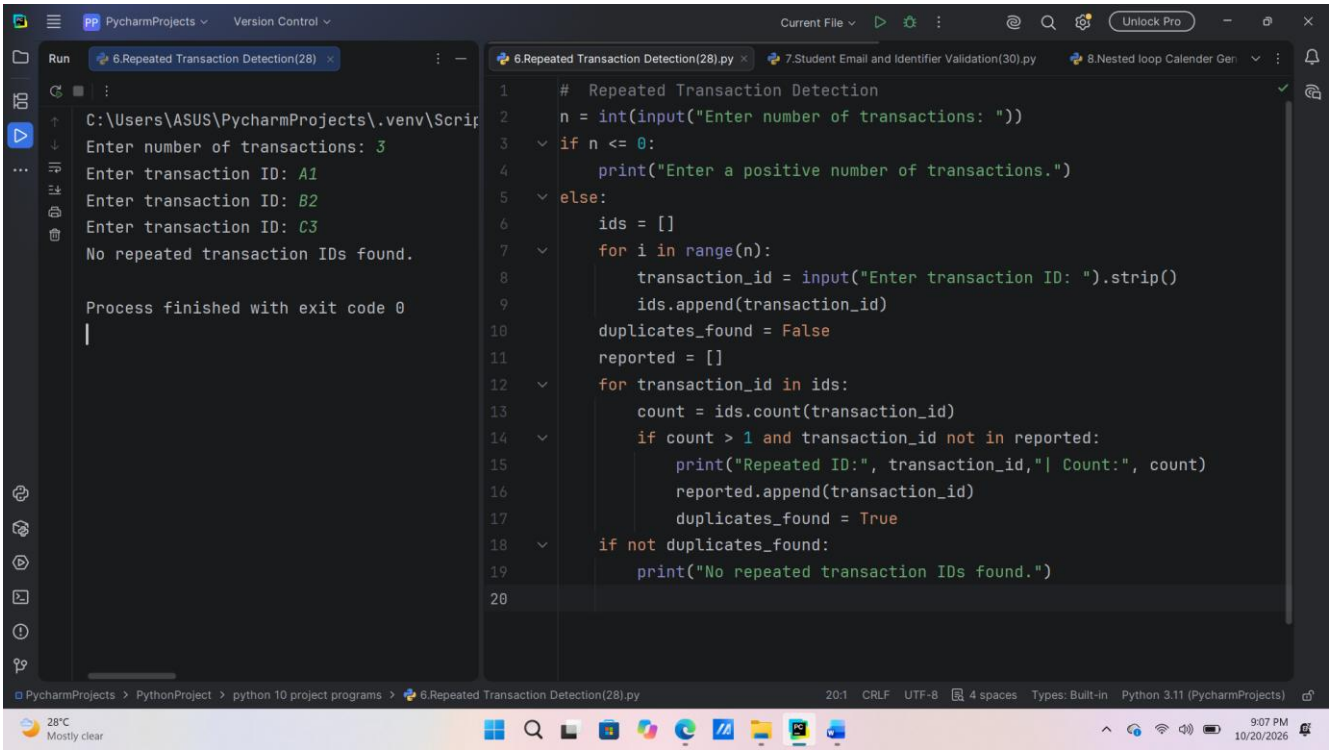
8. Test Cases

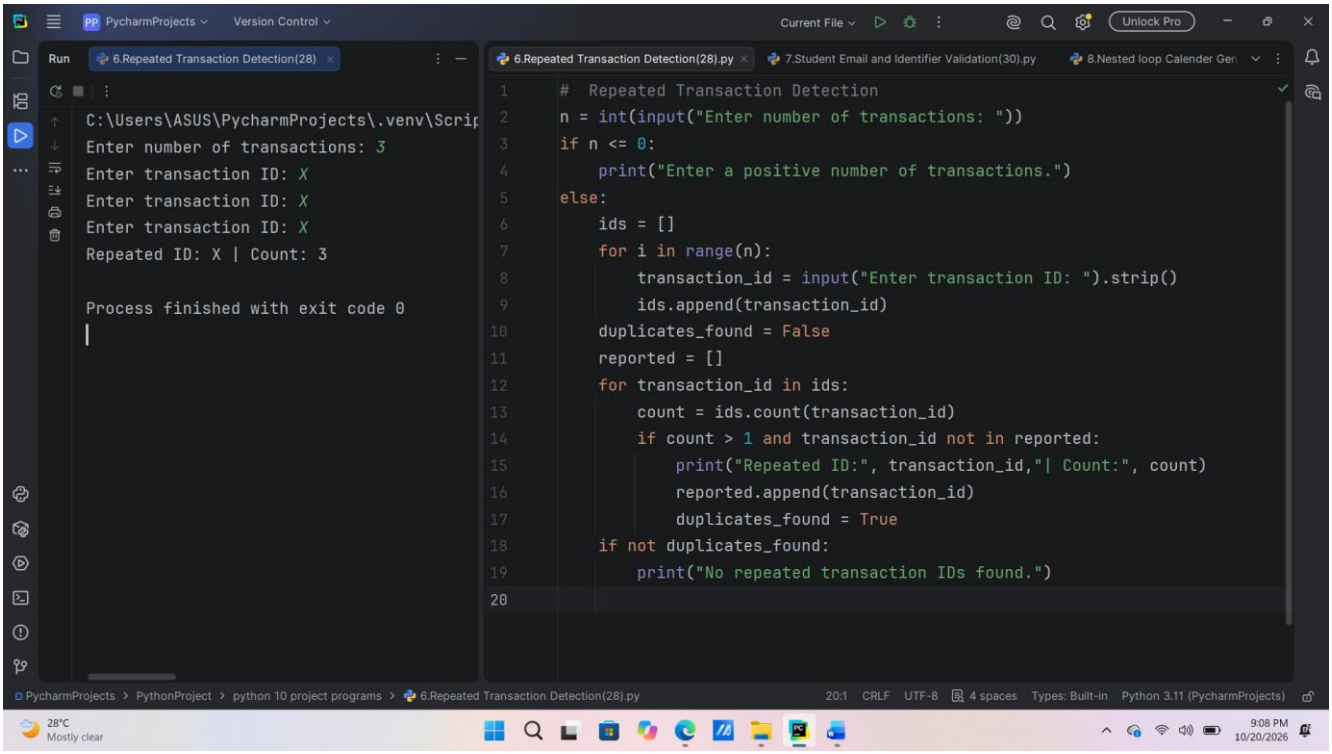
Transaction IDs	Expected result
T01, T02, T01	T01 repeated 2 times
A1, B2, C3	No repeated IDs
X, X, X	X repeated 3 times
Empty list	No repeated IDs / no data

9. Output Screenshots

Sample output :







10. Conclusion

The program uses a dictionary to count transaction IDs and reports repeated IDs that may need investigation.

Case Study 7: Student Email and Identifier Validation

Case Study No. 30 | Difficulty: Medium

1. Problem Statement

Write a program that validates student email addresses and student identifiers using simple, clearly defined rules, then reports whether each record is valid.

2. Objectives

- Check that an email has a non-empty name and domain separated by @.
- Check that the domain contains a dot and no spaces.
- Check student IDs against a simple alphanumeric format.
- Display validation results.

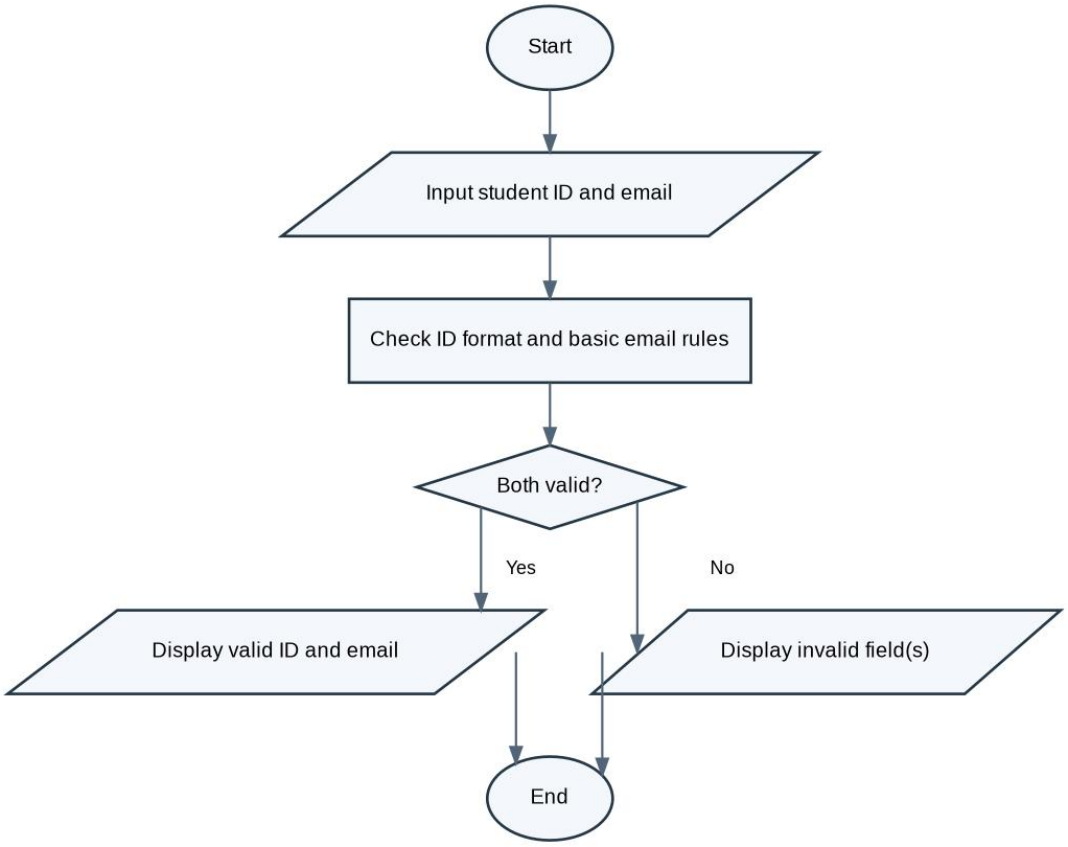
3. Assumptions

- This is basic classroom validation, not a complete email-standard validator.
- Student IDs must contain only letters and digits and be 5–12 characters long.
- Email must contain exactly one @, no spaces, and a dot in the domain.

4. Algorithm

1.Start.
2.Input student email and ID.
3.Validate email format.
4.Validate ID length and characters.
5.Report each result and overall status.
6.Stop.

5. Flowchart



6. Pseudocode

```
START
INPUT email, student_id
email_valid = email has exactly one @, no spaces,
              non-empty local part, and dot in domain
id_valid = length from 5 to 12 AND all characters alphanumeric
PRINT email result
PRINT ID result
IF email_valid AND id_valid: PRINT "Record valid"
ELSE: PRINT "Record needs correction"
STOP
```

7. Python Source Code

```
email = input("Enter student email: ").strip()
student_id = input("Enter student ID: ").strip()

parts = email.split("@")
email_valid = (
    len(parts) == 2
    and parts[0] != ""
    and parts[1] != ""
    and " " not in email
    and "." in parts[1]
    and not parts[1].startswith(".")
    and not parts[1].endswith(".")
)
id_valid = (
    student_id.startswith("STU")
    and student_id[3:].isdigit()
    and len(student_id) > 3
)
```

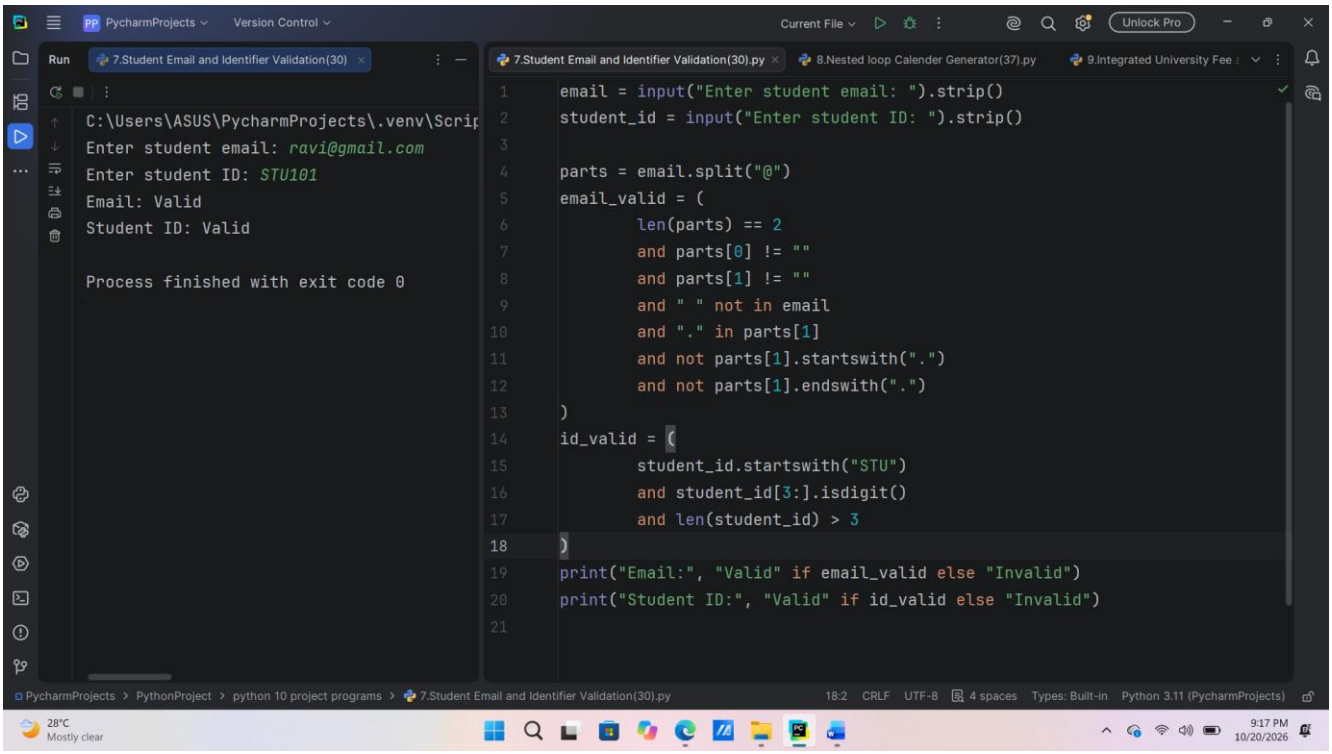
```
print("Email:", "Valid" if email_valid else "Invalid")
print("Student ID:", "Valid" if id_valid else "Invalid")
```

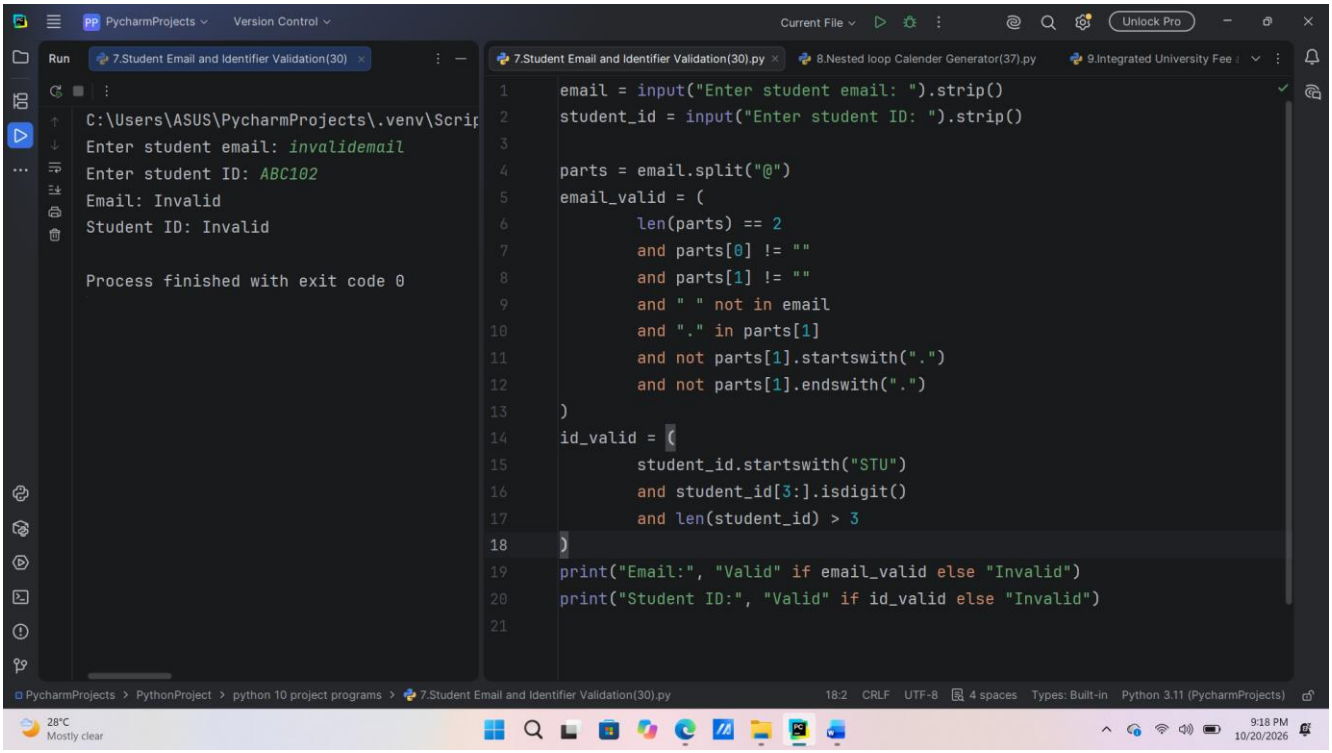
8. Test Cases

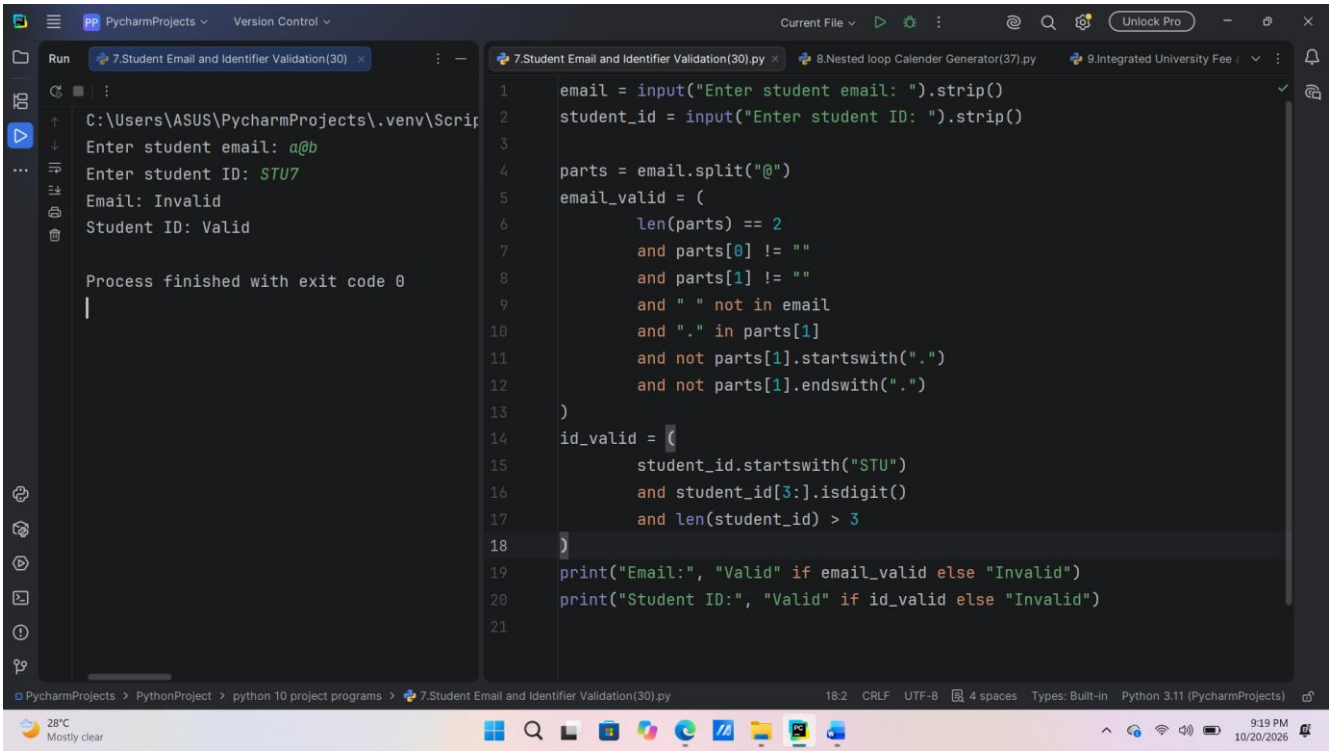
Email / Student ID	Expected result
ravi@gmail.com / STU101	Both valid
invalidemail / ABC102	Invalid email and ID
a@b / STU7	Email invalid under basic dot rule; ID valid
ravi @gmail.com / STU12	Email invalid due to space; ID valid

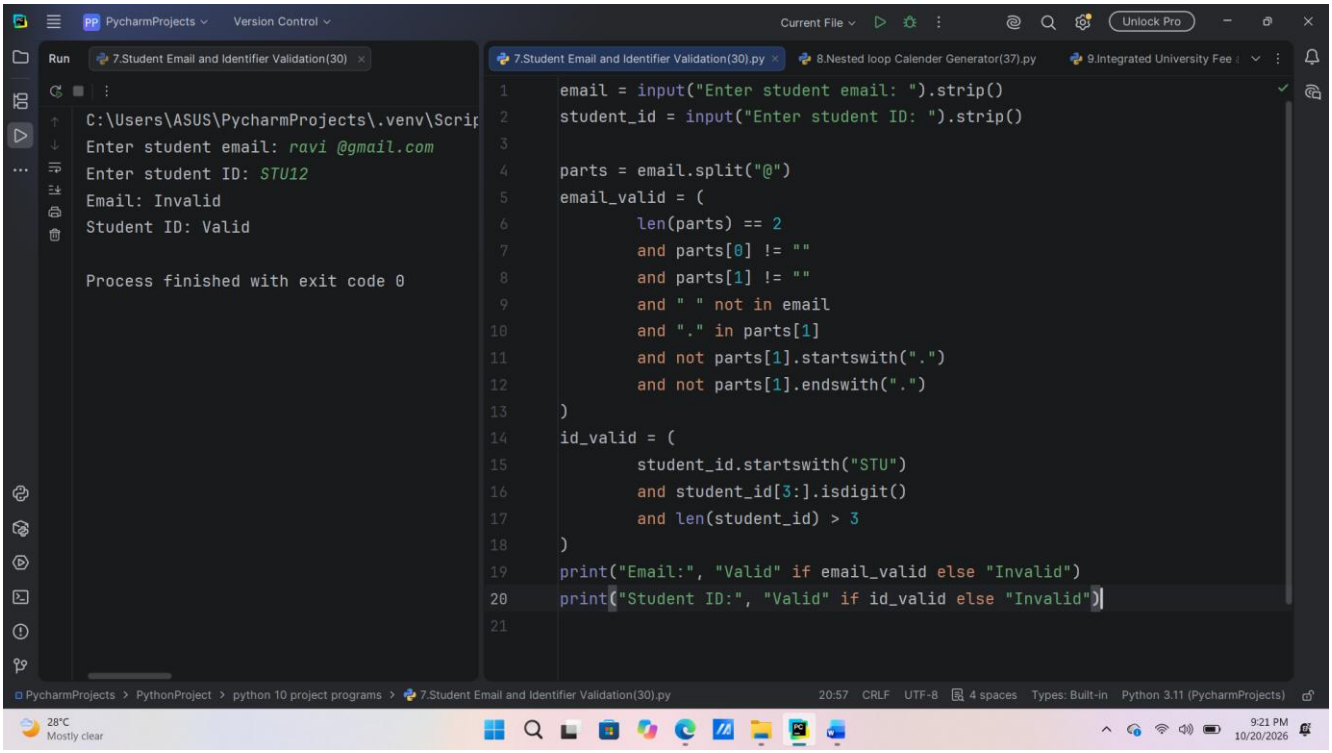
9. Output Screenshots

Sample output :




Run
7.Student Email and Identifier Validation(30)
C:\Users\ASUS\PycharmProjects\.venv\Scrip
Enter student email: invalidemail
Enter student ID: ABC102
Email: Invalid
Student ID: Invalid
Process finished with exit code 0
email = input("Enter student email: ").strip()
student_id = input("Enter student ID: ").strip()
parts = email.split("@")
email_valid = (
len(parts) == 2
and parts[0] != ""
and parts[1] != ""
and " " not in email
and "." in parts[1]
and not parts[1].startswith(".")
and not parts[1].endswith(".")
)
id_valid = (
student_id.startswith("STU")
and student_id[3:].isdigit()
and len(student_id) > 3
)
print("Email:", "Valid" if email_valid else "Invalid")
print("Student ID:", "Valid" if id_valid else "Invalid")


Run
7.Student Email and Identifier Validation(30)
C:\Users\ASUS\PycharmProjects\.venv\Scrip
Enter student email: a@b
Enter student ID: STU7
Email: Invalid
Student ID: Valid
Process finished with exit code 0
email = input("Enter student email: ").strip()
student_id = input("Enter student ID: ").strip()
parts = email.split("@")
email_valid = (
len(parts) == 2
and parts[0] != ""
and parts[1] != ""
and " " not in email
and "." in parts[1]
and not parts[1].startswith(".")
and not parts[1].endswith(".")
)
id_valid = (
student_id.startswith("STU")
and student_id[3:].isdigit()
and len(student_id) > 3
)
print("Email:", "Valid" if email_valid else "Invalid")
print("Student ID:", "Valid" if id_valid else "Invalid")



10. Conclusion

The program checks email and student-ID formats using basic validation rules and flags records that require correction.

Case Study 8: Nested Loop Calendar Generator

Case Study No. 37 | Difficulty: Medium

1. Problem Statement

Create a program that accepts a month and year, validates them, and prints a monthly calendar with Monday as the first weekday.

2. Objectives

- Accept the number of days in the month.
- Accept the starting weekday index, where Monday is 0 and Sunday is 6.
- Print dates in aligned weekly rows.
- Use nested loops to organize the calendar.

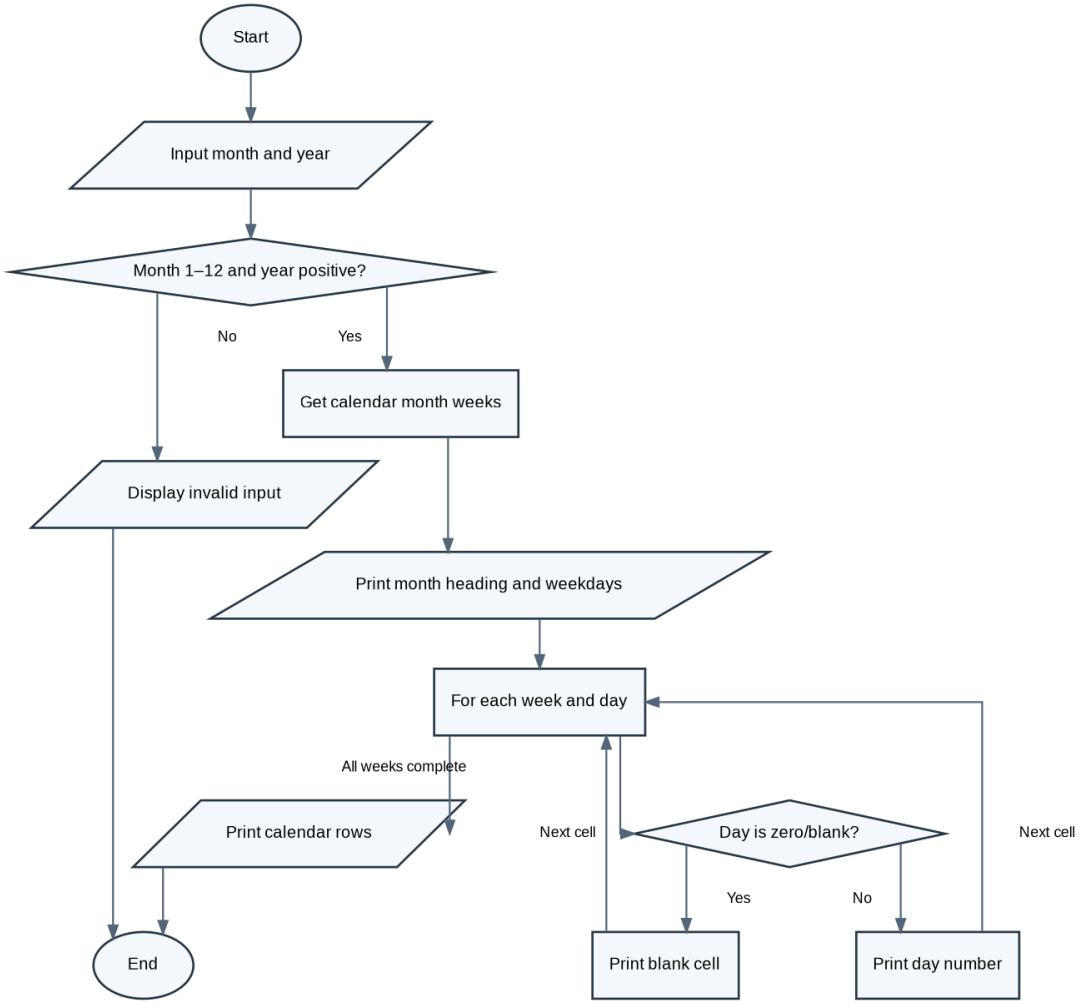
3. Assumptions

- The user supplies the correct day count for the month.
- The first weekday index is from 0 to 6.
- This simplified calendar does not calculate month/year dates automatically.

4. Algorithm

1.Start.
2.Input days and starting weekday.
3.Validate days and weekday index.
4.Print weekday headings.
5.Print initial blank spaces.
6.Loop through dates 1 to days, print each date in a fixed-width field, and start a new row every Sunday.
7.Stop.

5. Flowchart



6. Pseudocode

```
START
INPUT days, first_weekday
IF days < 28 OR days > 31 OR first_weekday < 0 OR first_weekday > 6
    PRINT "Invalid input"
ELSE
    PRINT weekday headings
    PRINT first_weekday blank cells
    FOR day = 1 TO days
        PRINT day in fixed-width cell
        IF (first_weekday + day) MOD 7 == 0: PRINT NEWLINE
    END FOR
END IF
STOP
```

7. Python Source Code

```
import calendar

month = int(input("Enter month (1-12): "))
year = int(input("Enter year: "))

if month < 1 or month > 12 or year <= 0:
    print("Invalid month or year.")
else:
```

```
print(f"\n{calendar.month_name[month]} {year}")
print("Mo Tu We Th Fr Sa Su")
weeks = calendar.monthcalendar(year, month)

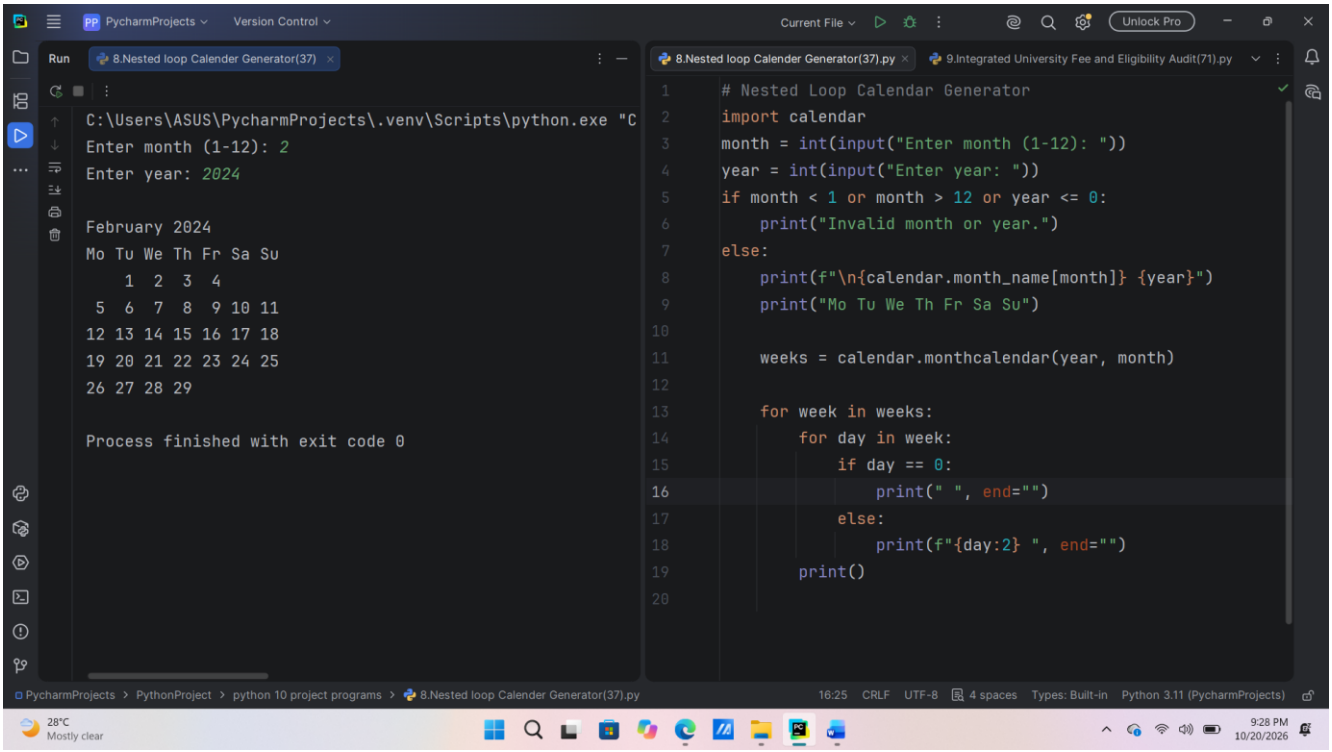
for week in weeks:
    for day in week:
        if day == 0:
            print("   ", end="")
        else:
            print(f"{day:2} ", end="")
    print()
```

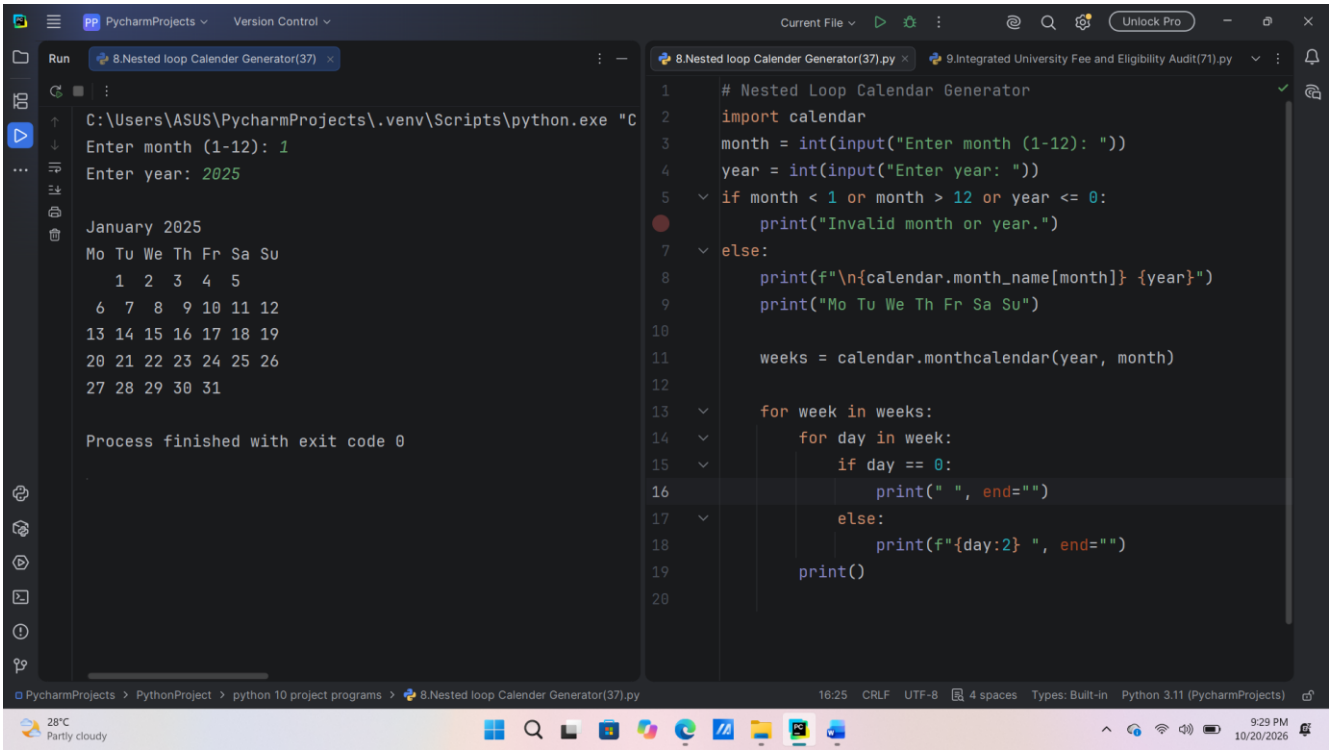
8. Test Cases

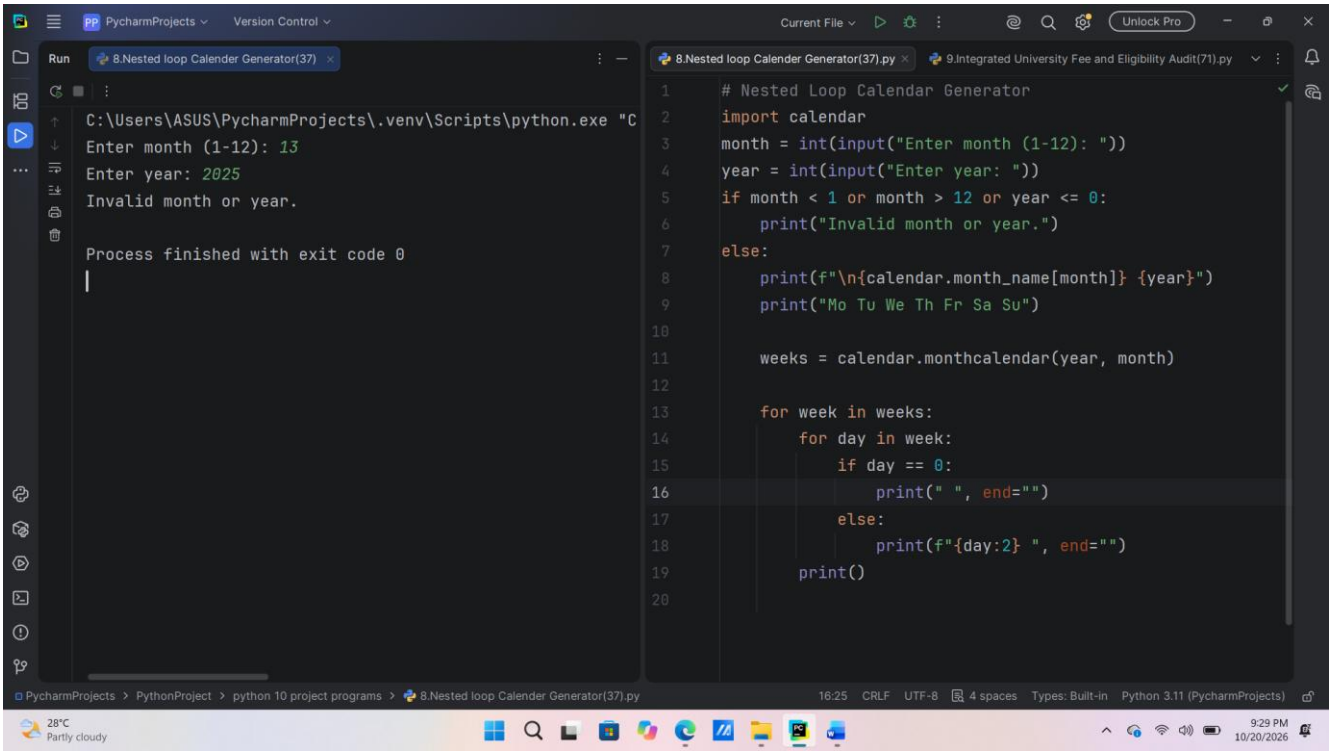
Month / year	Expected result
2 / 2024	February 2024 leap-year calendar
1 / 2025	January 2025 calendar
13 / 2025	Invalid month or year
2 / 0	Invalid month or year

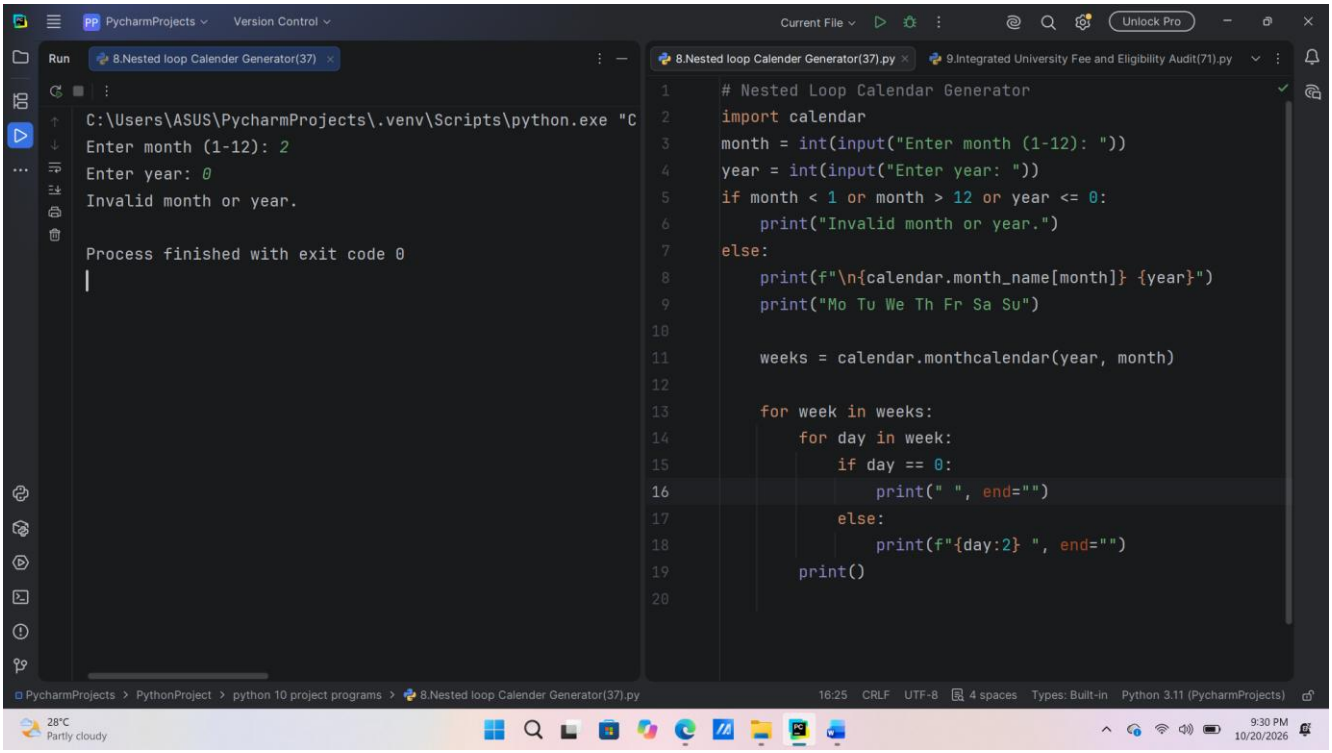
9. Output Screenshots

Sample output:




PycharmProjects
Version Control
Current File
Unlock Pro
Run
8.Nested loop Calender Generator(37)
C:\Users\ASUS\PycharmProjects\.venv\Scripts\python.exe "C
Enter month (1-12): 1
Enter year: 2025
January 2025
Mo Tu We Th Fr Sa Su
1 2 3 4 5
6 7 8 9 10 11 12
13 14 15 16 17 18 19
20 21 22 23 24 25 26
27 28 29 30 31
Process finished with exit code 0
8.Nested loop Calender Generator(37).py
9.Integrated University Fee and Eligibility Audit(71).py
Nested Loop Calendar Generator
import calendar
month = int(input("Enter month (1-12): "))
year = int(input("Enter year: "))
if month < 1 or month > 12 or year <= 0:
print("Invalid month or year.")
else:
print(f"\n{calendar.month_name[month]} {year}")
print("Mo Tu We Th Fr Sa Su")
weeks = calendar.monthcalendar(year, month)
for week in weeks:
for day in week:
if day == 0:
print(" ", end="")
else:
print(f"{day:2} ", end="")
print()
PycharmProjects > PythonProject > python 10 project programs > 8.Nested loop Calender Generator(37).py
16:25 CRLF UTF-8 4 spaces Types: Built-in Python 3.11 (PycharmProjects)
28°C
Partly cloudy
9:29 PM
10/20/2026


PycharmProjects
Version Control
Current File
Unlock Pro
Run
8.Nested loop Calender Generator(37)
C:\Users\ASUS\PycharmProjects\.venv\Scripts\python.exe "C
Enter month (1-12): 13
Enter year: 2025
Invalid month or year.
Process finished with exit code 0
8.Nested loop Calender Generator(37).py
9.Integrated University Fee and Eligibility Audit(71).py
Nested Loop Calendar Generator
import calendar
month = int(input("Enter month (1-12): "))
year = int(input("Enter year: "))
if month < 1 or month > 12 or year <= 0:
print("Invalid month or year.")
else:
print(f"\n{calendar.month_name[month]} {year}")
print("Mo Tu We Th Fr Sa Su")
weeks = calendar.monthcalendar(year, month)
for week in weeks:
for day in week:
if day == 0:
print(" ", end="")
else:
print(f"{day:2} ", end="")
print()
PycharmProjects > PythonProject > python 10 project programs > 8.Nested loop Calender Generator(37).py
16:25 CRLF UTF-8 4 spaces Types: Built-in Python 3.11 (PycharmProjects)
28°C
Partly cloudy
9:29 PM
10/20/2026



10. Conclusion

The program uses loops and weekday offsets to print a basic calendar layout for a user-specified month.

Case Study 9: Integrated University Fee and Eligibility Audit

Case Study No. 71 | Difficulty: Hard

1. Problem Statement

Create a program that calculates a student's pending university fee and checks examination eligibility using two conditions: the fee must be fully paid and attendance must be at least 75%.

2. Objectives

Read the student name, total fee, amount paid, and attendance percentage.

Validate fee and attendance values.

Calculate the pending fee.

Determine eligibility and explain any unmet condition.

3. Assumptions

Total fee must be greater than zero.

Fee paid must be between zero and total fee.

Attendance must be between 0 and 100.

In this simplified model, eligibility requires zero pending fee and attendance of at least 75%.

4. Algorithm

1.Start.

2.Input student name, total fee, fee paid, and attendance.

3.Validate the fee values and attendance range.

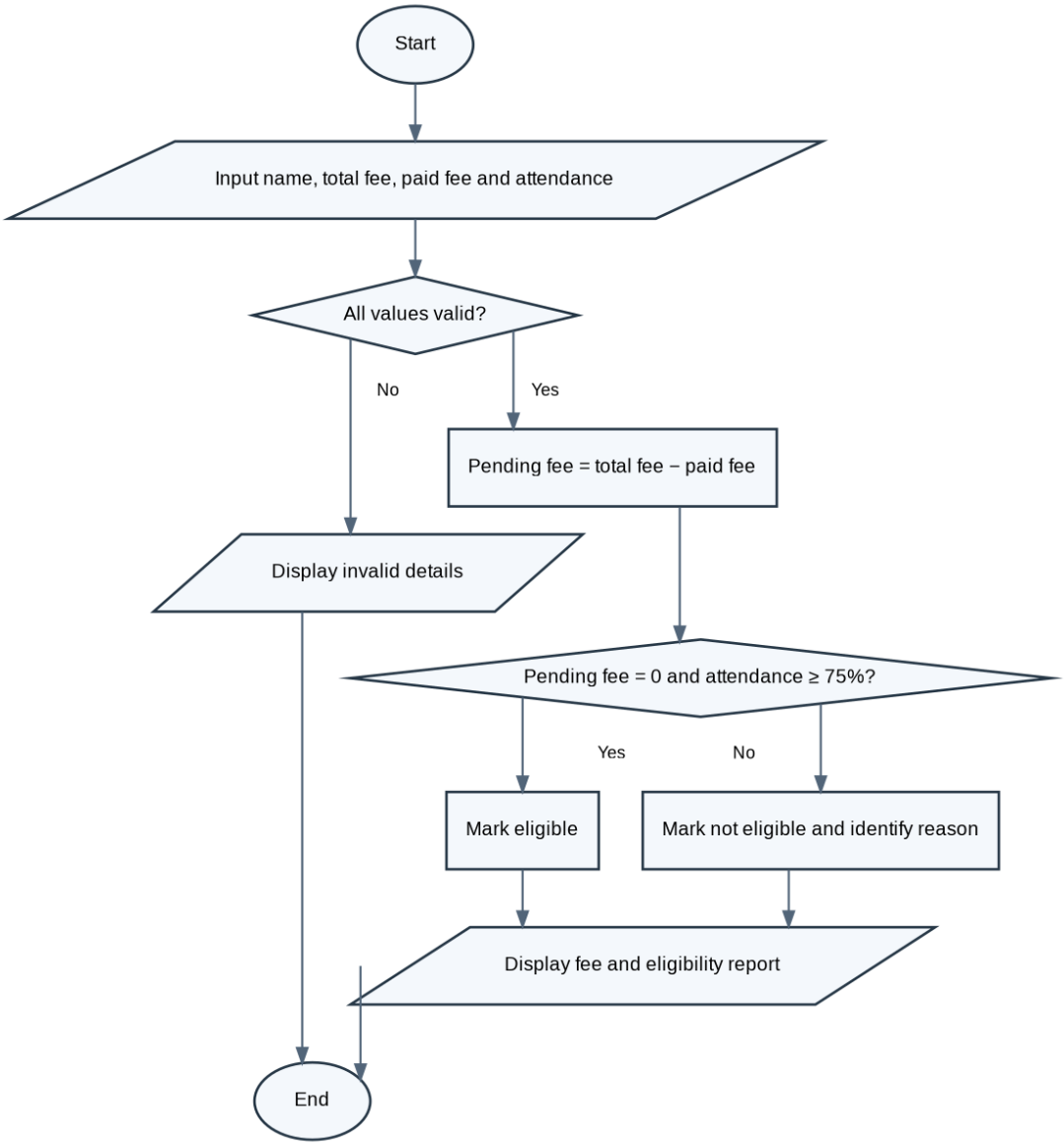
4.Calculate pending fee = total fee − fee paid.

5.If pending fee is zero and attendance is at least 75%, mark eligible; otherwise mark not eligible and display the reason.

6.Display the fee and eligibility report.

7.Stop.

5. Flowchart



6. Pseudocode

```
START
INPUT name, total_fee, fee_paid, attendance
IF total_fee <= 0 OR fee_paid < 0 OR fee_paid > total_fee OR attendance < 0 OR attendance > 100 THEN
   PRINT invalid student details
ELSE
   pending_fee = total_fee - fee_paid
   IF pending_fee == 0 AND attendance >= 75 THEN PRINT eligible
   ELSE PRINT not eligible and each applicable reason
   END IF
END IF
STOP
```

7. Python Source Code

```
student_name = input("Enter student name: ").strip()
total_fee = float(input("Enter total fee: "))
fee_paid = float(input("Enter fee paid: "))
```

```
attendance = float(input("Enter attendance percentage: "))

if (
    total_fee <= 0
    or fee_paid < 0
    or fee_paid > total_fee
    or attendance < 0
    or attendance > 100
):
    print("Invalid student details.")
else:
    pending_fee = total_fee - fee_paid
    print("\nStudent Name:", student_name)
    print("Total Fee:", total_fee)
    print("Fee Paid:", fee_paid)
  print(f"Pending Fee: {pending_fee:.2f}")
    print("Attendance:", attendance, "%")

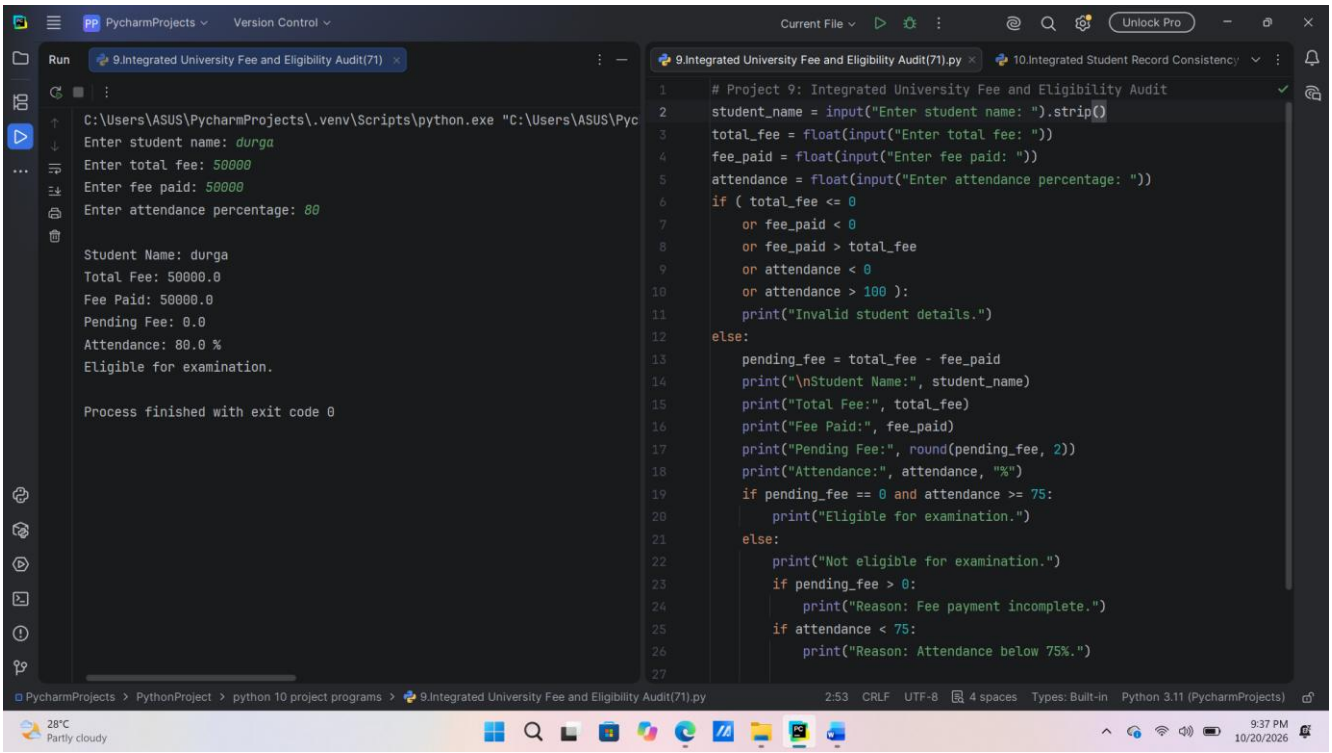
    if pending_fee == 0 and attendance >= 75:
        print("Eligible for examination.")
    else:
        print("Not eligible for examination.")
        if pending_fee > 0:
            print("Reason: Fee payment incomplete.")
        if attendance < 75:
            print("Reason: Attendance below 75%.")
```

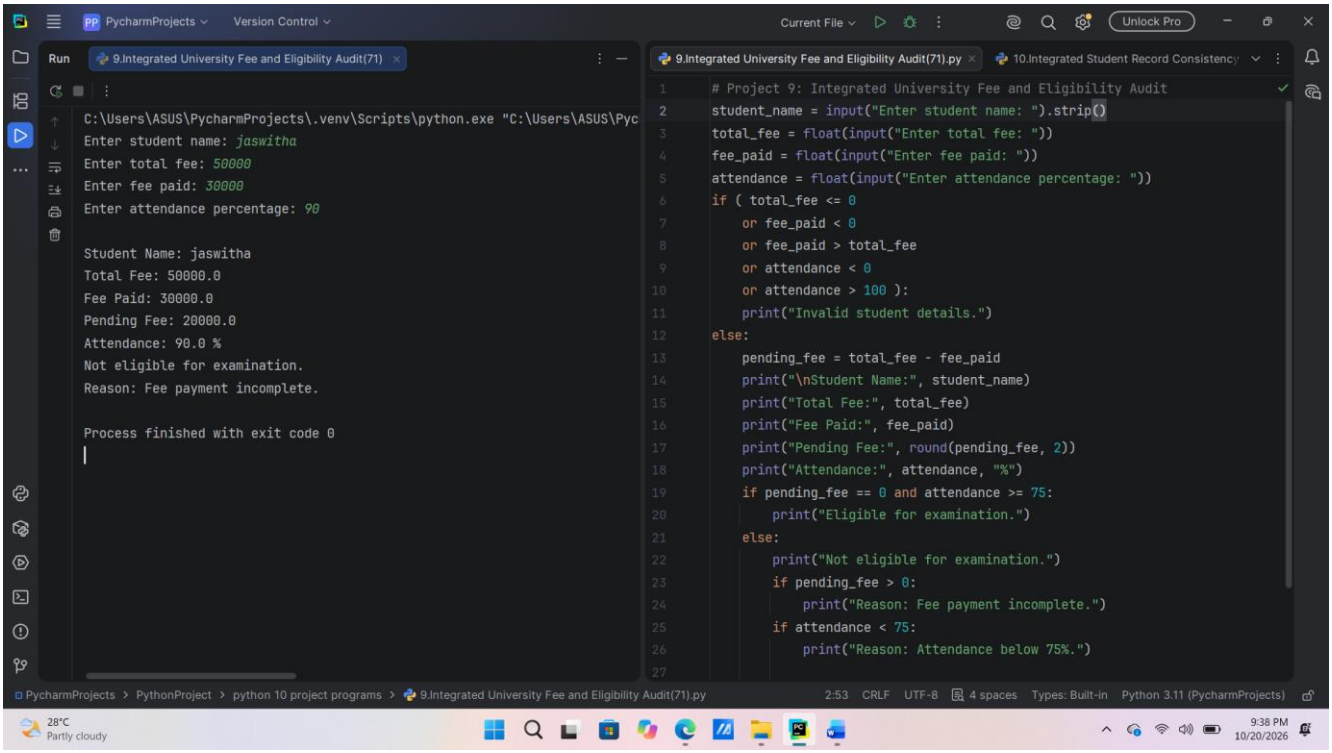
8. Test Cases

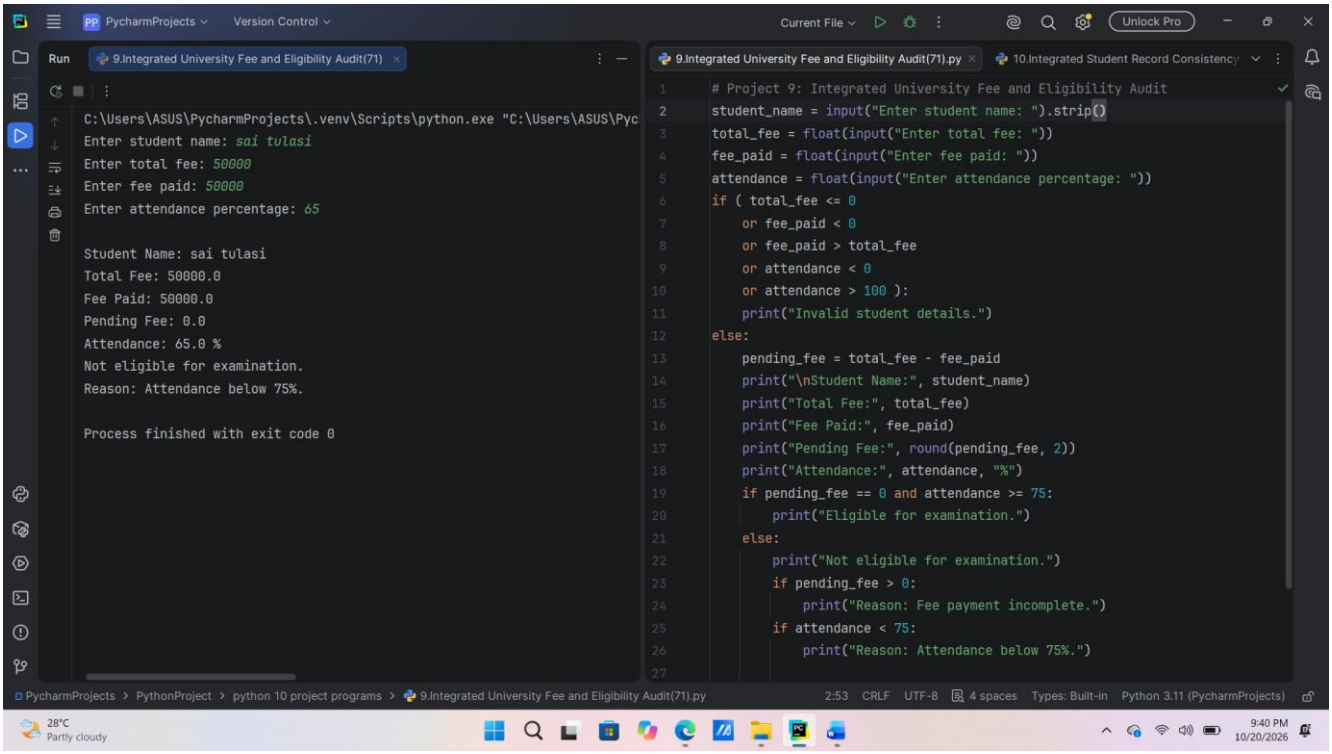
Total / paid / attendance	Expected result
50000 / 50000 / 80	Pending ₹0.00; eligible
50000 / 30000 / 90	Pending ₹20000.00; not eligible due to fee
50000 / 50000 / 65	Pending ₹0.00; not eligible due to attendance
1000 / 1200 / 110	Invalid student details

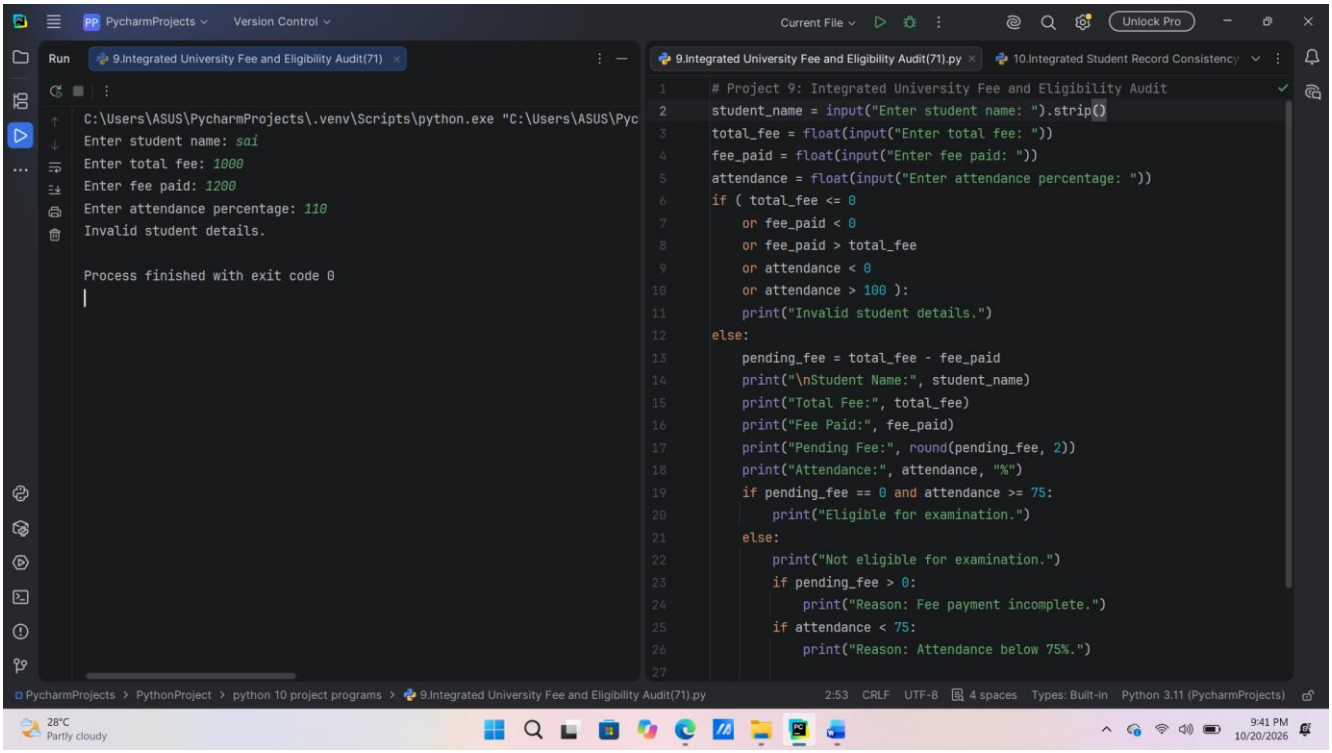
9. Output Screenshots

Sample output :




PycharmProjects
Version Control
Current File
Unlock Pro
Run
9.Integrated University Fee and Eligibility Audit(71)
C:\Users\ASUS\PycharmProjects\.venv\Scripts\python.exe "C:\Users\ASUS\Pyc
Enter student name: jaswitha
Enter total fee: 50000
Enter fee paid: 30000
Enter attendance percentage: 90
Student Name: jaswitha
Total Fee: 50000.0
Fee Paid: 30000.0
Pending Fee: 20000.0
Attendance: 90.0 %
Not eligible for examination.
Reason: Fee payment incomplete.
Process finished with exit code 0
9.Integrated University Fee and Eligibility Audit(71).py
10.Integrated Student Record Consistency
Project 9: Integrated University Fee and Eligibility Audit
student_name = input("Enter student name: ").strip()
total_fee = float(input("Enter total fee: "))
fee_paid = float(input("Enter fee paid: "))
attendance = float(input("Enter attendance percentage: "))
if (total_fee <= 0
or fee_paid < 0
or fee_paid > total_fee
or attendance < 0
or attendance > 100):
print("Invalid student details.")
else:
pending_fee = total_fee - fee_paid
print("\nStudent Name:", student_name)
print("Total Fee:", total_fee)
print("Fee Paid:", fee_paid)
print("Pending Fee:", round(pending_fee, 2))
print("Attendance:", attendance, "%")
if pending_fee == 0 and attendance >= 75:
print("Eligible for examination.")
else:
print("Not eligible for examination.")
if pending_fee > 0:
print("Reason: Fee payment incomplete.")
if attendance < 75:
print("Reason: Attendance below 75%.")
PycharmProjects > PythonProject > python 10 project programs > 9.Integrated University Fee and Eligibility Audit(71).py
2:53 CRLF UTF-8 4 spaces Types: Built-in Python 3.11 (PycharmProjects)
28°C
Partly cloudy
9:38 PM
10/20/2026


PycharmProjects
Version Control
Current File
Unlock Pro
Run
9.Integrated University Fee and Eligibility Audit(71)
C:\Users\ASUS\PycharmProjects\.venv\Scripts\python.exe "C:\Users\ASUS\Pyc
Enter student name: sai tulasi
Enter total fee: 50000
Enter fee paid: 50000
Enter attendance percentage: 65
Student Name: sai tulasi
Total Fee: 50000.0
Fee Paid: 50000.0
Pending Fee: 0.0
Attendance: 65.0 %
Not eligible for examination.
Reason: Attendance below 75%.
Process finished with exit code 0
9.Integrated University Fee and Eligibility Audit(71).py
10.Integrated Student Record Consistency
Project 9: Integrated University Fee and Eligibility Audit
student_name = input("Enter student name: ").strip()
total_fee = float(input("Enter total fee: "))
fee_paid = float(input("Enter fee paid: "))
attendance = float(input("Enter attendance percentage: "))
if (total_fee <= 0
or fee_paid < 0
or fee_paid > total_fee
or attendance < 0
or attendance > 100):
print("Invalid student details.")
else:
pending_fee = total_fee - fee_paid
print("\nStudent Name:", student_name)
print("Total Fee:", total_fee)
print("Fee Paid:", fee_paid)
print("Pending Fee:", round(pending_fee, 2))
print("Attendance:", attendance, "%")
if pending_fee == 0 and attendance >= 75:
print("Eligible for examination.")
else:
print("Not eligible for examination.")
if pending_fee > 0:
print("Reason: Fee payment incomplete.")
if attendance < 75:
print("Reason: Attendance below 75%.")
PycharmProjects > PythonProject > python 10 project programs > 9.Integrated University Fee and Eligibility Audit(71).py
2:53 CRLF UTF-8 4 spaces Types: Built-in Python 3.11 (PycharmProjects)
28°C
Partly cloudy
9:40 PM
10/20/2026



10. Conclusion

The program combines fee computation and attendance checking into one audit report, using validation to prevent unreasonable inputs.

Case Study 10: Integrated Student Record Consistency Checker

Case Study No. 75 | Difficulty: Hard

1. Problem Statement

Create a program that reads multiple student records and checks student ID format, basic email format, marks range, and duplicate student IDs. It reports all detected issues for each record.

2. Objectives

Store multiple student records in a nested list.

Validate student IDs and email-like strings.

Check marks are within 0–100.

Detect duplicate student IDs and report all errors for each record.

3. Assumptions

The number of records must be positive.

Student IDs must start with STU and be followed by one or more digits.

Email validation is basic: exactly one @, no spaces, and a dot in the domain.

Marks must be between 0 and 100.

Every record sharing a duplicate ID is flagged.

4. Algorithm

1.Start and input the number of records.

2.Reject a non-positive record count.

3.For each student, input ID, name, email, and marks; store the record in a nested list.

4.For each record, validate ID, email, and marks.

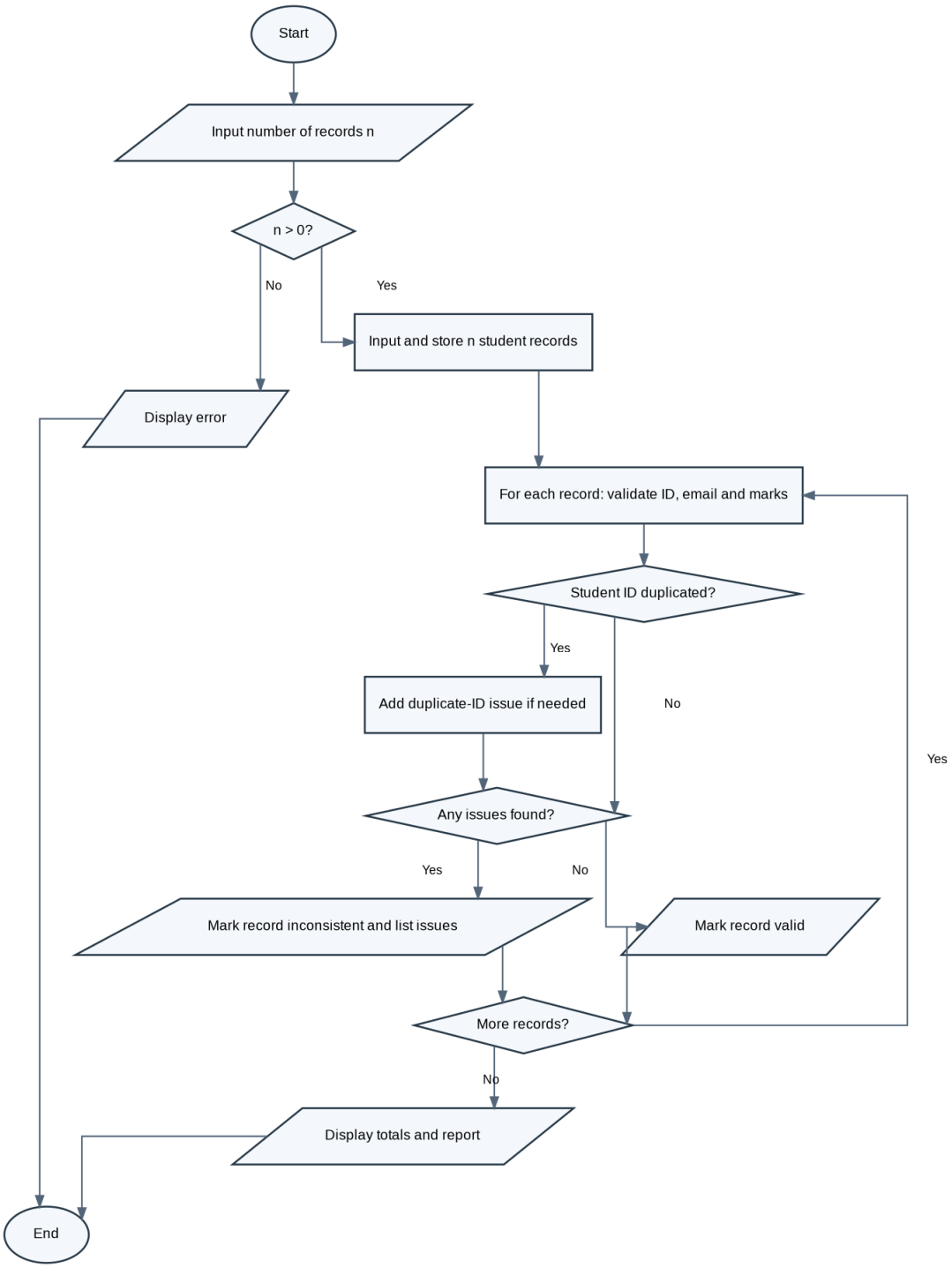
5.Count records using the same student ID and flag duplicates.

6.Print the record status and every issue found.

7.Print totals for records, records with errors, and records without errors.

8.Stop.

5. Flowchart



6. Pseudocode

```
START
INPUT n
IF n <= 0 THEN PRINT error
ELSE
   INPUT n student records and store in a list
```

```
    FOR each record
      validate ID, email, and marks; collect all issues
      count records with the same ID; flag duplicates
      print valid status or every issue
    END FOR
    print total records and error totals
END IF
STOP
```

7. Python Source Code

```
n = int(input("Enter number of student records: "))

if n <= 0:
    print("Enter a positive number of records.")
else:
    records = []
    for i in range(n):
        print("\nStudent", i + 1)
        student_id = input("Enter student ID: ").strip()
        name = input("Enter student name: ").strip()
        email = input("Enter student email: ").strip()
        marks = float(input("Enter marks (0-100): "))
        records.append([student_id, name, email, marks])

    error_count = 0
    print("\nStudent Record Validation Report")
    for record in records:
        student_id, name, email, marks = record
        errors = []

        if not (student_id.startswith("STU") and student_id[3:].isdigit()):
            errors.append("Invalid student ID")

        parts = email.split("@")
        email_valid = (
            len(parts) == 2 and parts[0] != "" and parts[1] != ""
            and " " not in email and "." in parts[1]
            and not parts[1].startswith(".") and not parts[1].endswith(".")
        )
        if not email_valid:
            errors.append("Invalid email")
        if marks < 0 or marks > 100:
            errors.append("Invalid marks")

        id_count = 0
        for other_record in records:
            if other_record[0] == student_id:
                id_count += 1
        if id_count > 1:
            errors.append("Duplicate student ID")

        print("\nName:", name)
        print("Student ID:", student_id)
        if errors:
            print("Status: Inconsistent")
            for error in errors:
                print("-", error)
            error_count += 1
        else:
            print("Status: Valid")

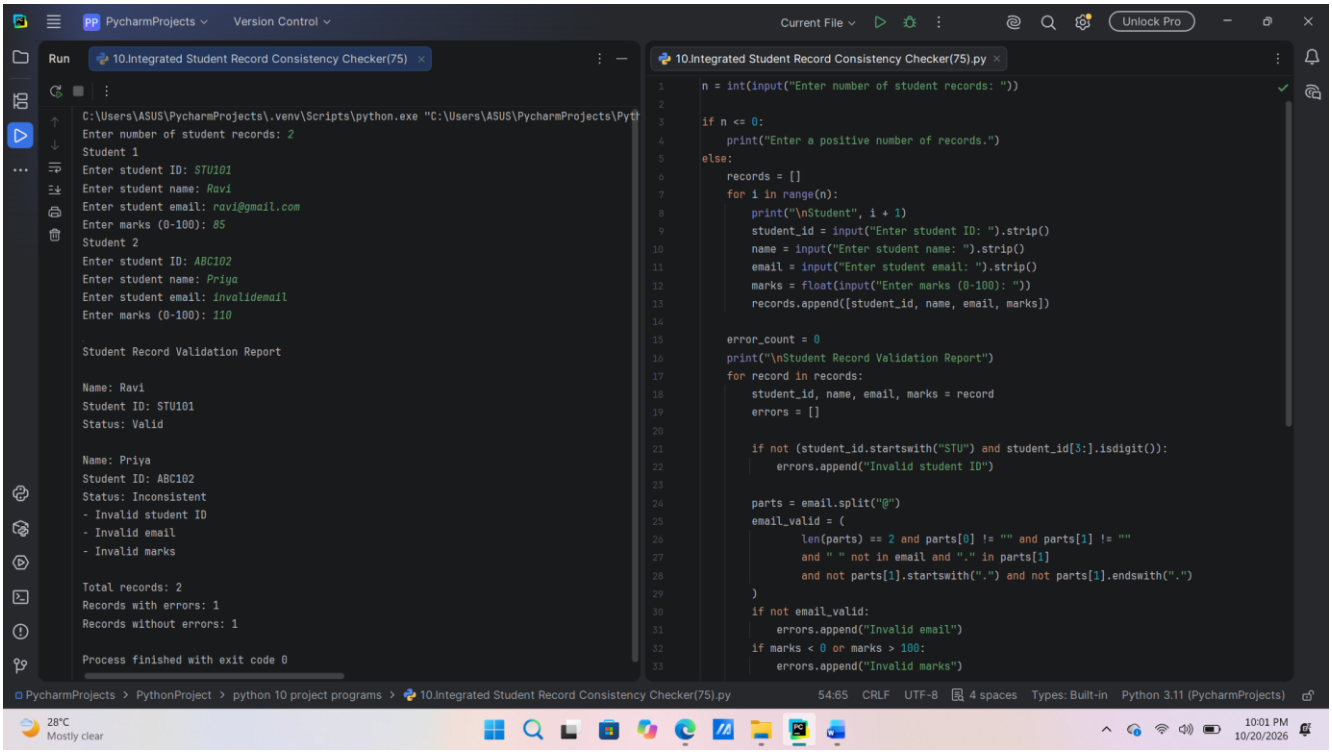
    print("\nTotal records:", len(records))
    print("Records with errors:", error_count)
    print("Records without errors:", len(records) - error_count)
```

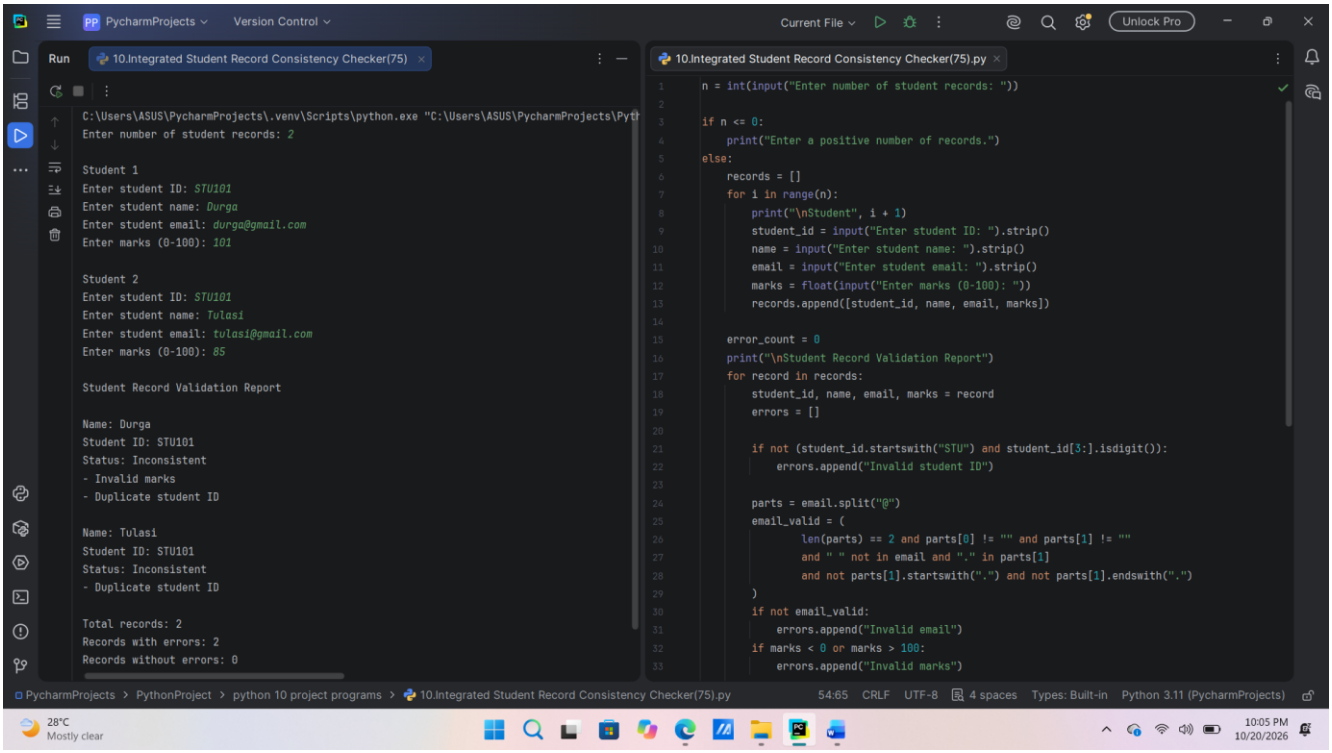
8. Test Cases

Input record(s)	Expected result
STU101, Ravi, ravi@gmail.com, 85	Valid if ID unique
ABC102, Priya, invalidemail, 110	Invalid ID, email, marks
Two records use STU101	Both flagged for duplicate ID
Marks=-1 or 101	Invalid marks
Number of records=0	Positive-number-of-records error

9. Output Screenshots

Sample output :





10. Conclusion

The program validates multiple student fields together and reports all consistency problems, making record review easier.